Fall26_SLS775. Natural Language Processing in Corpus Research

Hakyung Sung, Second Language Studies, University of Hawaii at Manoa
Last updated: October 30, 2026

Contents

1 Introduction to NLP for corpus linguistics **4**
1.1 What is corpus linguistics? . . . 5
1.2 What is NLP? . . . 7

2 NLP in learner corpus research **11**
2.1 What is learner corpus research? . . . 11
2.1.1 Learner corpus data . . . 11
2.1.2 Methodological approaches and research designs . . . 12
2.1.3 Linguistic foci . . . 13
2.1.4 Contributions to SLA . . . 13
2.1.5 Contributions to L2 teaching and assessment . . . 14
2.2 How can LCR be connected to NLP? . . . 15
2.2.1 Common NLP (pre-)processes . . . 15
2.2.2 Challenges in evaluating NLP accuracy . . . 19

3 Corpus design and selection **22**
3.1 Sampling and representativeness . . . 22
3.1.1 An operational definition of a corpus . . . 23
3.1.2 Size. . . . 23
3.1.3 Principled sampling. . . . 23
3.1.4 Representativeness. . . . 24
3.1.5 Reusing existing corpora. . . . 25
3.1.6 Representativeness and balance . . . 25
3.2 Different types of corpora . . . 26
3.3 Different types of information . . . 26
3.4 Different ways to process corpora . . . 28

4 Unit of analysis **31**
4.1 Lexical units . . . 31
4.1.1 Larger units in vocabulary pedagogy . . . 32
4.1.2 Smaller units in vocabulary research . . . 32
4.1.3 Effects on vocabulary research and pedagogy . . . 32
4.2 Tokenization . . . 34
4.2.1 Tokens and types . . . 34
4.2.2 Written and spoken Language . . . 35
4.2.3 Word boundaries across languages . . . 35
4.2.4 Morphemes as possible units . . . 35
4.2.5 Characters and Unicode . . . 36
4.2.6 Subword tokenizers . . . 36

5 Word vectors **38**
5.1 Distributional semantics . . . 38

5.2 Vector semantics . . . 40
5.2.1 Simple count-based embeddings . . . 40
5.2.2 Cosine for measuring similarity . . . 41
5.2.3 Word2Vec . . . 42

6 Neural networks **47**
6.1 A single neural unit . . . 47
6.2 From weighted sum to activation . . . 48
6.3 Common activation functions . . . 49
6.4 Connection to logistic regression . . . 50
6.5 How neural networks learn . . . 50

7 Dependency parsing and tagging **55**
7.1 Basics of UD . . . 55
7.1.1 Dependency structures . . . 55
7.1.2 Words as the basic units . . . 56
7.1.3 Part-of-speech categories . . . 57
7.1.4 Grammatical relations . . . 58
7.2 Some implications of UD . . . 58
7.2.1 UD as a corpus resource . . . 60
7.2.2 UD and learner corpora . . . 60
7.2.3 Corpus-based language comparison . . . 61
7.2.4 Comparability does not guarantee representativeness . . . 61

8 Measuring constructs in SLA **63**
8.1 From complexity as an umbrella term to distinct constructs . . . 64
8.1.1 Complexity in the CAF framework . . . 64
8.1.2 Different meanings of complexity . . . 64
8.2 Defining complexity and difficulty . . . 65
8.2.1 Complexity . . . 65
8.2.2 Difficulty . . . 67
8.3 Interactions among complexity, difficulty, development, and proficiency . . . 69

9 Linguistic annotations **71**
9.1 What is corpus annotation? . . . 71
9.2 Benefits of annotation . . . 71
9.2.1 Efficient retrieval . . . 71
9.2.2 Reusability of annotations . . . 73
9.3 Principles of good annotation practice . . . 74
9.3.1 Preserve and separate the original data . . . 74
9.3.2 Document the annotation explicitly . . . 74
9.3.3 Prefer interpretable and consensual categories . . . 75
9.3.4 Respect established practice without treating it as immutable . . . 75
9.4 Evaluating annotation quality . . . 75
9.4.1 Validity: are the categories linguistically meaningful? . . . 75
9.4.2 Accuracy: how often are reference labels recovered? . . . 76
9.4.3 Consistency: do annotators apply the scheme in the same way? . . . 77

10 Semantic role labeling **78**
10.1 Semantic roles . . . 78
10.2 Diathesis alternations . . . 79

10.3 PropBank 80
10.4 FrameNet 80
10.5 Semantic role labeling 81
10.5.1 Feature-based approaches 82
10.5.2 Neural approaches: SRL as sequence labeling 83
10.6 Beyond semantic role labeling 85

1 Introduction to NLP for corpus linguistics

References

- Dunn (2022). *Natural language processing for corpus linguistics*. Cambridge University Press. Chapter 1.1: Scaling Up Corpus Linguistics. [Section 1 of these notes]
- Gries (2026). Corpus linguistics and psycholinguistics. In *International encyclopedia of language and linguistics* (3rd ed., pp. 412–416). Elsevier. [Section 1.1]
- Meurers (2020), Natural language processing and language learning. *Encyclopedia of applied linguistics*, 4193-4205. [Section 1.2]

Corpus linguistics and natural language processing (NLP) both investigate language through collections of naturally occurring linguistic data. However, the two fields have traditionally differed in their primary goals. Corpus linguistics generally begins with a linguistic question and uses corpus data to describe patterns of language use, while NLP develops computational methods for automatically analyzing or generating human language. NLP for corpus linguistics brings these perspectives together: computational models are used to identify, annotate, measure, and compare linguistic patterns in corpora at a scale that would difficult to achieve through manual analysis alone.

This combination has become increasingly important as the size and diversity of available language data have expanded. Early electronic corpora, such as the Brown Corpus, contained approximately one million words, whereas contemporary corpora may contain billions of words. Such datasets create new opportunities for investigating linguistic variation, but they also make exclusively manual analysis impractical. Computational methods therefore allow corpus linguists to scale up their analyses while maintaining an explicit and, in principle, reproducible analytical procedure.

One way to organize computational problems in corpus linguistics is to distinguish between two broad types of analysis: (1) *categorization* and (2) *comparison*. Categorization involves assigning a predefined label to a linguistic unit. For example, a corpus linguist may classify a word by its part of speech, determine whether a sentence instantiates a particular grammatical construction, or identify the genre to which a document belongs. Comparison, by contrast, involves assessing the degree of similarity or relatedness between linguistic units. A researcher may examine whether two words are semantically related, whether two sentences convey similar meanings, or whether two texts share stylistic characteristics.

These two types of analysis are not rigidly tied to particular machine-learning algorithms. Note that in *supervised* learning, a model is trained on examples that have already been assigned *target labels* (in other words, annotated tags). The model learns the relationship between the input

features and the expected output and then applies that relationship to new data. In *unsupervised* learning, by contrast, the data are not provided with predefined target labels. The model instead identifies patterns, groupings, or relationships within the data.

Categorization is often implemented through supervised learning because researchers typically define the categories that a model is expected to predict. For example, a researcher may provide texts labeled as academic writing, news, or conversation and train a classifier to assign new texts to one of these registers. However, categories can also be explored through unsupervised methods. A clustering algorithm might group texts according to their linguistic similarities, after which the researcher examines whether the resulting clusters correspond to recognizable registers.

Comparison is often conducted using representations or similarity measures that do not require predefined class labels. For example, a researcher may represent two sentences as vectors and calculate their cosine similarity to estimate how closely related their meanings are. However, comparison can also involve supervised learning. A model may be trained on sentence pairs that have been labeled with degrees of semantic similarity and then used to predict the similarity of new sentence pairs.

1.1 What is corpus linguistics?

Corpus linguistics is an *empirical* approach to the study of language based on the systematic analysis of collections of linguistic data known as *corpora*. At the most basic technical level, a corpus may consist of multiple machine-readable text files accompanied by information about the texts, speakers, and contexts from which the data were collected (i.e., metadata). However, a corpus is not simply any large collection of texts. Its value as a research resource depends on how systematically the data were selected, organized, and documented. This distinction also helps clarify the difference between linguistic *data* and a *corpus*: whereas data may refer broadly to any collection of linguistic observations, a corpus is a deliberately constructed and documented dataset designed to support systematic analysis of a particular population or domain of language use.

A corpus is generally designed to represent a particular population or domain of language use, such as a group of speakers, a register, a genre, a topic, a historical period, or a language variety. Its sampling scheme should therefore reflect relevant sources of variability within the population it is intended to represent. For example, a corpus of contemporary spoken English may sample speakers from different regions, age groups, and communicative settings. Claims based on a corpus are consequently limited by the population and contexts represented in its design. We will return to this issue in our discussion of corpus *representativeness*.

Corpus linguistics is primarily concerned with the distribution of linguistic forms and functions. Researchers examine how frequently linguistic elements occur, the contexts in which they occur, the other elements with which they co-occur, and how these distributions vary across speakers, texts, registers, or time periods. These distributional patterns can be summarized through

measures (e.g., token frequency, association strength).

Corpora are commonly used in psycholinguistics as sources of quantitative measures derived from the distributional patterns of language use. These measures can serve as operationalizations of psycholinguistic constructs that are not directly obserable. In other words, corpus statistics may not be able to measure cognitive representations or processes directly, but they quantify aspects of an individual's likely linguistic experience that may be relevant to how language is (learned and) used. For example, one of the most basic corpus measures is *token frequency*, or the number of times a particular linguistic form or pattern occurs. Token frequency can be related to *entrenchment*, the process through which repeated exposure strengthens a linguistic representation and allows it to become established as a cognitive routine.[1] Similarly, the frequency with which two linguistic elements occur together may contribute to the entrenchment of that combination. Corpus data also make it possible to quantify *contingency*, or the extent to which the occurrence of one linguistic element predicts the occurrence of another. Such relationships can be investigated through analyses of collocation and/or collostruction, often using conditional probabilities or association measures. For example, researchers may examine whether a particular verb occurs with a given construction more frequently than would be expected from the independent frequencies of the verb and the construction. Strong contingency may facilitate the learning of form–form or form–meaning associations because the occurrence of one element provides reliable information about another.

One important advantage of corpus data is their potential for ecological validity.[2] Experimental studies are often conducted under highly controlled conditions using carefully selected and sometimes decontextualized stimuli. Such control is necessary for establishing causal relationships, but the resulting language may not always reflect the distributions encountered in ordinary communication. Balanced experimental designs may also expose participants to linguistic forms in proportions that differ substantially from those found in everyday language use. Corpus data complement experimental evidence by documenting how language is distributed in naturally occurring, less experimentally constrained communicative contexts.

At the same time, corpus data present several analytical challenges. Naturally occurring language is often noisy and heterogeneous, and linguistic variables are frequently highly intercorrelated. Contexts may vary considerably both within and across speakers, making it difficult to isolate the influence of any single factor. Corpus distributions are also typically highly imbalanced and approximately Zipfian: a small number of linguistic elements occur very frequently, whereas most occur only rarely. Consequently, corpus frequencies and associations must be interpreted in light of corpus composition, sampling decisions, dispersion, and the number of relevant opportunities for occurrence.

[1] For example, a word, phrase, or grammatical construction that occurs frequently in language use may be recognized or processed more readily than a less frequently encountered alternative.

[2] Ecological validity refers to "the degree to which a given research context is indicative of, or can be generalized to, real-world environments" (Schmuckler, 2026). More details here: https://oecs.mit.edu/pub/1y8kpgak

Corpus linguistics is therefore not simply the automated counting of words in large text collections. It is a research approach that connects systematically sampled language data, quantitative descriptions of linguistic distributions, and theoretically motivated interpretations of language use. Its strength lies in allowing researchers to investigate how linguistic patterns emerge across large numbers of contextualized observations while remaining attentive to the limitations of the data and the procedures used to analyze them.

1.2 What is NLP?

NLP is concerned with the automated analysis and generation of human language. NLP systems process written or spoken language in order to identify linguistic properties, extract information, make predictions, or produce new language. Common NLP tasks include dividing texts into words and sentences, assigning part-of-speech (POS) categories, identifying syntactic relationships, measuring semantic similarity, translating between languages, classifying documents, answering questions, and generating text. Speech recognition and speech synthesis are also closely related to NLP, although speech processing is sometimes treated as a separate subfield.

NLP can operate at different levels of linguistic analysis. At the word level, a system may identify lemmas, morphological features, or POS tags. At the sentence level, it may analyze grammatical structure, identify errors, or represent meaning. At the text level, it may measure cohesion, classify genre, estimate proficiency, or generate a summary. NLP therefore provides methods for transforming unstructured language into representations that can be systematically searched, counted, compared, and modeled.

This role makes NLP particularly important for corpus linguistics. A corpus may contain thousands or millions of texts that cannot be analyzed manually in their entirety. NLP tools can automatically annotate these texts and extract linguistic features relevant to a research question. For example, a researcher may use an NLP pipeline to identify verbs and their subjects, calculate lexical and syntactic complexity measures for individual texts, or represent texts as vectors for classification and comparison. NLP therefore supports the scalability of corpus analysis. However, the validity of the resulting analysis depends on several factors: the capabilities and limitations of the computational method used (e.g., a simple count-based system vs. Transformer architecture), the characteristics of the corpus to which it is applied (e.g., edited standard-language texts from news sources vs. L2 learner language), and the extent to which the computational output provides a valid operationalization of the linguistic construct under investigation (e.g., text length vs. grammatical complexity).

In the context of language learning, NLP has two broad applications. The first is the analysis of *learner language*: words, sentences, and texts produced by language learners. The second is the analysis of *language for learners*: texts and other language materials that are selected, adapted, or generated to support learning.

The first broad application includes areas such as intelligent language tutoring systems, au-

tomatic language assessment, and grammatical error detection. An intelligent language tutoring system may analyze a learner's response in order to provide individualized feedback, select an appropriate subsequent activity, or update a model of the learner's developing abilities. NLP is necessary in such systems because it is generally impossible to anticipate every response that a learner might produce and manually specify the appropriate feedback for each possible string. Instead, the system must abstract away from the exact wording of the response and identify more general properties of its form, meaning, and appropriateness. This requirement creates a distinctive challenge for NLP. Many general-purpose NLP systems are designed to remain robust when input contains errors/unexpected forms. A parser, for instance, is typically expected to produce a syntactic analysis even when a sentence is not fully well formed. In a language tutoring system, however, the unexpected form may itself be the information that the system needs to identify. The system must determine not only that a learner' s response differs from an expected target but also what kind of difference has occurred, what it may reveal about the learner' s developing language system, and what feedback would help the learner understand the problem. High correction accuracy alone is therefore not always sufficient. A system that produces an acceptable corrected sentence without identifying the linguistic source of the problem may provide limited insight into the learner' s underlying difficulty and limited support for learning.

NLP is also used to annotate and analyze learner corpora with a goal to investigate broader patterns of development and variation. NLP can support error annotation, identify overuse/underuse of linguistic forms, calculate measures of complexity, accuracy, and fluency, and examine relationships between linguistic features and proficiency. It can also help researchers investigate cross-linguistic influence by identifying patterns associated with learners' first-language backgrounds. By automating at least part of this process, NLP makes it possible to analyze larger learner corpora and connect theoretical questions more directly to relevant instances in the data. This application closely reflects the role of NLP in corpus linguistics introduced in Section 1 and will be a central focus throughout this course.

The second broad application involves analyzing language that is presented to learners. For example, NLP can help identify reading materials that contain particular vocabulary or grammatical structures, estimate the difficulty or readability of texts, highlight target forms, provide links to definitions or grammatical explanations, and generate exercises or assessment items from existing materials. Such applications can give learners more individualized access to language resources and allow instructional materials to be selected according to their proficiency, goals, or current learning needs.

We briefly discussed NLP within the context of applied linguistics, but the field can be defined more broadly when viewed from the perspectives of computer science and artificial intelligence. The following definition, adapted from Vajjala et al. (2020, p. 6), highlights this broader perspective:

NLP is an interdisciplinary field at the intersection of computer science, artificial in-

telligence, and linguistics. It focuses on developing computational systems that can process, analyze, and understand human language. Although NLP was historically concentrated in academic institutions and research laboratories, recent advances have enabled its widespread adoption in areas such as retail, healthcare, finance, law, marketing, and human resources. Several developments have contributed to this expansion:

1. the growing availability of accessible NLP tools, techniques, and application programming interfaces (APIs);
2. improvements in generalizable and interpretable approaches to complex tasks, including open-domain dialogue and question answering;
3. increased investment in interactive consumer technologies that use language as a primary means of communication;
4. the availability of open-source datasets and standardized benchmarks; and
5. the development of datasets and language-specific models for languages beyond English and other widely digitized languages.

Chapter Summary

Corpus linguistics and NLP both use collections of language data, but they have traditionally emphasized different goals. Corpus linguistics begins with a question about language use and examines the distribution of linguistic forms and functions in (systematically constructed) corpora. NLP develops computational methods for processing, analyzing, and generating language. NLP for corpus linguistics brings these perspectives together by using computational methods to annotate, search, measure, categorize, and compare linguistic patterns across datasets that may be too large for entirely manual analysis.

Activity: Brainstrom your project for this course

You might not have a concrete research question yet. Start with a topic that you are currently studying or may be interested in studying throughout the course.

1. Briefly describe your research area or a research question that interests you. What aspect of language, language use, or language learning would you like to investigate?
2. What kind of language data could help you investigate this question? For example, would you examine learner writing, classroom interaction, interviews, social media posts, published texts, assessment responses, or another type of data?
3. Could this dataset be considered a corpus? What information would you need about its speakers, writers, texts, tasks, or contexts in order to interpret it appropriately?
4. What linguistic feature, pattern, or construct would you examine in the data? Examples might include vocabulary use, grammatical constructions, errors, syntactic complexity, semantic similarity, interactional features, or differences across learner groups.
5. Would your analysis mainly involve *categorization*, *comparison*, or both? Explain your choice using one example from your research topic.
6. What part of the analysis might NLP help automate? What would you still want a researcher to examine or verify manually?
7. Identify potential challenge(s). For example, could the corpus be unrepresentative, could the NLP tool perform poorly on the language data, or could the computational measure fail to capture the intended linguistic construct?

Group report: Select one research example from your group and briefly explain (1) the research question, (2) the proposed language data, and (3) one way that NLP could support the analysis.

2 NLP in learner corpus research

References

- Granger (2026). Corpus-Based Research in Second Language Acquisition. In *International encyclopedia of language and linguistics* (3rd ed.). Elsevier. [Section 2.1]
- Kyle (2021). Natural language processing for learner corpus research. *International Journal of Learner Corpus Research*, *7*(1), 1-16. [Section 2.2]
- Kyle and Sung (2026). Tagging and parsing. In *International encyclopedia of language and linguistics* (3rd ed.). Elsevier. [Section 2.2]

2.1 What is learner corpus research?

Learner corpus research (LCR) is a field that uses learner corpora to investigate the characteristics and development of learner language. Learner corpus research has generally pursued two closely connected objectives. The first is *theoretical*: LCR seeks to contribute to second language acquisition (SLA) research by providing more detailed descriptions of learner language and by improving our understanding of the factors that influence it. These factors may include learners' first-language backgrounds, proficiency levels, learning contexts, task characteristics, and cognitive or affective variables. The second objective is *pedagogical*: findings from learner corpora can be used to develop teaching materials, assessment practices, and language-learning tools that more accurately address learners' needs.

2.1.1 Learner corpus data

Early learner corpora focused predominantly on L2 English. Since then, learner corpora have been developed for a much wider range of target languages. An overview of existing resources is available through the Learner Corpora around the World[3] database. Nevertheless, English remains disproportionately represented, particularly in corpora of academic writing.

A basic distinction can be made between *raw* and *annotated* learner corpora. Raw learner corpora contain textual or transcribed spoken data in their original, relatively unprocessed form (usually in `.txt` format). They may include learner metadata (e.g., first language, age, proficiency, instructional context, or task type - usually in a separated file), but the linguistic data themselves have not been systematically enriched with additional analytical information. Annotated learner corpora include information about particular linguistic features (could be in `.xlm` or `.conllu`[4] formats). Depending on the research purpose, annotation may cover parts of speech, morphological structure, syntactic dependencies, semantic categories, pragmatic functions, discourse features, pronunciation, disfluencies, or characteristics of spoken interaction. Some forms of annotation can be produced automatically through tools such as part-of-speech taggers, morphological analyz-

[3] https://uclouvain.be/en/research-institutes/ilc/cecl/learner-corpora-around-the-world.html
[4] https://universaldependencies.org/format.html

ers, and syntactic parsers. When these tools perform with sufficiently high accuracy, they make it possible to process much larger collections of learner language than would be feasible through entirely manual analysis.

Other forms of annotation continue to require substantial human judgment. Error annotation is an important example. Researchers may identify a problematic form, categorize the type of error, and sometimes reconstruct the learner's intended target form. However, both error detection and error categorization are inherently interpretive. An annotator must decide whether an unusual form is actually erroneous, what the learner may have intended, and which linguistic category best describes the problem. These judgments may be especially difficult when a sentence contains several interacting errors. For this reason, manually annotated learner corpora should be accompanied by detailed annotation manuals that define the categories and provide examples of difficult cases. Researchers should also report intra-rater or inter-rater reliability statistics when the annotation depends on human judgment. Reliability assessment does not eliminate subjectivity, but it makes the annotation process more transparent and provides evidence regarding the consistency of the resulting data.

2.1.2 Methodological approaches and research designs

The characteristics of learner corpus data have led to the development of methodological approaches specifically designed for investigating learner language. One of the best-known approaches is *Contrastive Interlanguage Analysis* (CIA; Granger 2015). CIA originally focused on comparisons between learner language and corresponding native-speaker or expert varieties. For example, researchers might compare the frequencies of particular words, grammatical constructions, collocations, or discourse markers in learner essays and expert academic writing.

Most learner corpus research has used a *cross-sectional* design, in which data are collected from different learners at one point in time. Researchers may then compare groups representing different proficiency levels or instructional stages. Cross-sectional designs are relatively practical because they do not require researchers to follow the same individuals for an extended period.

A *longitudinal* design, by contrast, collects repeated samples from the same learners over time. Longitudinal learner corpora are less common because they require sustained access to participants and must address issues such as participant attrition, changes in instructional context, and variation in elicitation tasks. Nevertheless, longitudinal learner corpus research is increasing because it provides more direct evidence of individual development.

A third approach is sometimes described as *pseudo-longitudinal*. In this design, researchers collect data from cohorts of learners at different proficiency levels at approximately the same point in time. The resulting differences do not demonstrate that particular individuals moved through the observed stages. However, they can provide detailed descriptions of different points along an assumed interlanguage continuum. This approach is popular in learner corpus research and

can inform the development of teaching materials or assessment resources designed for particular proficiency levels.

2.1.3 Linguistic foci

Learner corpus research investigates a broad range of linguistic phenomena, although some areas have received considerably more attention than others. A survey of the Learner Corpus Bibliography found that (1) lexical phenomena were examined in approximately 65% of studies, followed by (2) grammar at 46%, (3) discourse at 30%, and (4) pragmatics at 10%. Note that these categories are not mutually exclusive, so an individual study may contribute to more than one area.

Within lexical research, phraseology has been particularly prominent. Researchers have examined learners' use of collocations, lexical bundles, formulaic sequences, and associations between words. These studies may investigate whether learners use a narrower range of combinations, rely heavily on a small number of familiar expressions, or produce combinations that are grammatically acceptable but uncommon in the relevant register.

Grammatical learner corpus research includes topics such as tense and aspect, article use, modality, complementation, and syntactic construction use. Discourse and pragmatic studies frequently investigate features such as stance markers, engagement markers, connectors, discourse-organizing expressions, and strategies for expressing politeness or evaluation.

Most learner corpus studies have used written rather than spoken data. Academic writing has received especially strong attention, partly because written texts are easier to collect, store, and annotate, and partly because advanced academic writing creates important pedagogical needs. This emphasis has contributed valuable knowledge about L2 writing, but it also means that conversational language, informal registers, interactional development, and lower-proficiency production remain comparatively underrepresented.

2.1.4 Contributions to SLA

One of the original objectives of learner corpus research was to contribute to SLA theory. For a considerable period, however, the theoretical grounding of LCR remained relatively weak. Many early studies were primarily descriptive, reporting frequencies or differences without clearly connecting those observations to theories of language acquisition. This partly explains the initially limited uptake of learner corpus findings in the broader SLA community.

The situation has changed substantially (see Tracy-Ventura and Paquot, 2021). Learner corpus data now have a more established role in research on L2 acquisition, although corpus-based production data remain less central than experimental data (e.g., comprehension measures, acceptability judgments, reaction times, and online processing data). Learner corpora cannot directly reveal all of the cognitive processes underlying language acquisition. They can, however, provide large and diverse collections of naturally occurring or task-elicited learner production.

The scale and diversity of learner corpus data can make it possible to test whether findings based on a limited sample generalize across learner populations. For example, Murakami and Alexopoulou (2016) used large-scale learner data to challenge claims concerning a universal order of acquisition for L2 English morphemes. Their findings demonstrated that learners' first-language backgrounds substantially influenced the observed acquisition patterns. This illustrates how learner corpus evidence can qualify conclusions that may appear universal when they are based on only a small number of learner groups.

Learner corpus research has also made important contributions to the study of *linguistic complexity*. Corpus-based studies have helped researchers *operationalize* complexity by developing systematic measures of grammatical, syntactic, and lexical complexity. These measures have been used to examine how complexity develops over time and varies according to proficiency, L1, register, genre, and task complexity (e.g., Biber et al., 2020; Kyle et al., 2024; Sung and Kyle, 2025). A closely related area involves *predictive* studies. In addition to describing group-level differences, these studies examine which factors are associated with learners' linguistic choices or learning outcomes. For example, researchers may investigate which lexical, syntactic, phraseological, or morphological features predict human judgments of writing or speaking proficiency. Such analyses can inform theories of L2 development, but correlations between linguistic features and proficiency scores should not be interpreted as direct evidence of acquisition or causal relationships.

2.1.5 Contributions to L2 teaching and assessment

From its beginnings, learner corpus research has been strongly committed to improving L2 teaching through more learner-focused materials and pedagogical practices. Many of these resources have been designed for higher-intermediate or advanced learners, whose difficulties may involve subtle restrictions on usage, register, phraseology, and discourse organization rather than only basic grammatical errors.

One major area of application concerns *accuracy*. Learner corpus data can be used to identify recurrent grammatical errors, lexical infelicities, problematic collocations, and non-target-like phraseological patterns. These findings can then support materials intended to raise learners' awareness of common difficulties and improve grammatical, lexical, and phraseological accuracy. One example is the Louvain English for Academic Purposes Dictionary, or LEAD (Granger and Paquot, 2015, 2022). This web-based academic writing resource draws on analyses of academic texts produced by learners from several L1 backgrounds. It highlights cross-disciplinary academic words that learners frequently misuse, overuse, or underuse. It also presents their phraseological patterning, including common collocations and lexical bundles. The resource demonstrates how learner corpus findings can be translated into pedagogical support that goes beyond simple word definitions.

Learner corpora can also be incorporated into *data-driven learning* (DDL). In DDL activities, learners examine corpus examples and identify linguistic patterns for themselves, often with guid-

ance from the teacher. Learner-corpus-based DDL may expose students to examples of successful learner production, recurrent difficulties, or alternative revisions. Preliminary studies suggest that this form of instruction can be effective and is often received positively by students and teachers (Hartle, 2023). However, the direct use of L2 data in DDL remains relatively limited. More research is needed to determine which types of learner data are most useful, how examples should be selected and presented, and under what conditions learner-corpus-based DDL improves learning.

Learner corpus evidence may also inform proficiency-level descriptions, teaching sequences, and assessment criteria. For example, pseudo-longitudinal analyses can identify features that become more frequent, diverse, accurate, or sophisticated at different proficiency levels. These patterns may contribute to resources associated with frameworks (e.g., Common European Framework of Reference for Languages [CEFR]). However, corpus frequencies should not be converted directly into proficiency descriptors without considering task, register, learner population, and the communicative function of the feature.

2.2 How can LCR be connected to NLP?

At this point, we might have a good understanding that NLP broadly refers to the use of computational methods to analyze human language automatically (not synonymous with artificial intelligence, machine learning, or statistics, although it may draw on these approaches). NLP tools of varying levels of complexity have played an important role in the development of corpus linguistics generally and learner corpus research in particular. Relatively simple tools include concordancers, which retrieve all occurrences of a search word or expression and display them in their surrounding contexts, allowing researchers to identify and compare recurring linguistic patterns. More complex tools, such as part-of-speech (POS) taggers and syntactic parsers, automatically enrich corpus texts with grammatical and syntactic information. The following section first introduces several NLP processes commonly used to annotate and analyze learner corpora.

2.2.1 Common NLP (pre-)processes

Tokenization. Tokenization (i.e.g, the division of a text into word-like units called *tokens*) is typically the very first stage of an NLP pipeline. Although tokenization may appear straightforward in languages in which words are generally separated by spaces, even an English orthographic word may contain multiple linguistically meaningful components. Tokenization becomes more challenging in some languages (e.g., Korean), in which individual space-delimited units may regularly contain multiple morphemes. Decisions about token and morpheme boundaries can therefore become an important source of variation in corpus analysis.

Example: Tokenization in English and Korean

English sentence

She walked to school.

Word-level tokens

She |walked |to |school |. (5 tokens)

Morpheme-level units

she |walk |-ed |to |school |. (6 tokens)

(*Note.* Punctuation counted as a separate token)

Korean sentence

저는	학교에	갑니다.
저 + 는	학교 + 에	가 + ㅂ니다
I + topic marker	*school + locative marker*	*go + declarative ending*

'*I go to school.*'

Space-delimited units

저는 |학교에 |갑니다 |. (4 tokens)

Morpheme-level units

저 |는 |학교 |에 |가 |ㅂ니다 |. (7 tokens)

Lemmatization. Lemmatization reduces inflected word forms to a shared dictionary form or lemma. For example, *walks*, *walked*, and *walking* may be analyzed as forms of the lemma *walk*. Lemmatization is widely used to calculate lexical frequencies and diversity, although it is not necessary or preferable for every language or research question. Combining forms at the lemma level may obscure meaningful morphological variation, particularly when inflectional choices are themselves central to the analysis.

Example: Lemmatization and word-type counts

Example sentence

She *walks* to school. She *walked* home. She enjoys *walking*.

Word-form sequence

she |walks |to |school |she |walked |home |she |enjoys |walking

Lemmatized sequence

she |walk |to |school |she |walk |home |she |enjoy |walk

Count	Before lemmatization	After lemmatization
Tokens	10	10
Types	8	6

Word-form types:

{*she*, ***walks***, *to*, *school*, ***walked***, *home*, *enjoys*, ***walking***} = 8

Lemma types:

{*she*, ***walk***, *to*, *school*, *home*, *enjoy*} = 6

Tagging. Tagging commonly refers to the assignment of grammatical part-of-speech (POS) labels to words or tokens in a text. POS tagsets can be classified as either *coarse-grained* or *fine-grained*. Coarse-grained tagsets distinguish broad grammatical categories without encoding many distinctions within each category. For example, the Universal POS tagset (UPOS; Nivre et al., 2020) consists of 17 tags representing major word classes, including adjectives (`ADJ`), adverbs (`ADV`), nouns (`NOUN`), and verbs (`VERB`). Although UPOS distinguishes proper nouns from common nouns, it generally provides limited information about grammatical subclasses or inflectional forms within each word class.

Fine-grained tagsets encode more detailed, and often language-specific, distinctions within these broad POS categories. The Penn Treebank tagset (Marcinkiewicz, 1994; Santorini, 1990), for instance, contains 36 POS tags. Whereas UPOS assigns all common nouns the single tag `NOUN`, the Penn Treebank tagset distinguishes singular or mass nouns (`NN`) from plural nouns (`NNS`). The difference is even more apparent for verbs. UPOS generally assigns lexical verbs the single tag `VERB`, while the Penn Treebank tagset distinguishes six inflectional forms: base form (`VB`), past tense (`VBD`), gerund or present participle (`VBG`), past participle (`VBN`), non-third-person singular present (`VBP`), and third-person singular present (`VBZ`). The Sejong Korean tagset is a specialized tag set for tagging Korean morphemes.[5]

Figure 1 illustrates how an example sentence from the English Web Treebank (Bies et al., 2012; Silveira et al., 2014) is represented using the coarse-grained UPOS tagset and the more fine-grained Penn Treebank tagset.

	1	*2*	*3*	*5*	*6*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
Token	His	artworks	were	selected	and	exhibited	in	the	British	Museum	in	2002	.
UPOS tags	PRON	NOUN	AUX	VERB	CCONJ	VERB	ADP	DET	PROPN	PROPN	ADP	NUM	PUNCT
Penn tags	PRP$	NNS	VBD	VBN	CC	VBN	IN	DT	NNP	NNP	IN	CD	.

Figure 1: A comparison of UPOS tags vs. Penn Treebank tags. Adapted from Kyle and Sung (2026)

[5]See examples here: https://nlpxl2korean.github.io/UD-KSL/xpos/

Parsing. *Parsing* refers to the automatic analysis and annotation of syntactic structure in a text. Two major approaches are commonly used: (1) *constituency parsing* and (2) *dependency parsing*. Although both approaches represent relationships among words and larger syntactic units, they differ in how sentence structure is organized and described.

(1) A *constituency parse* represents a sentence or utterance as a hierarchical tree composed of nested phrases and clauses. Words are grouped into syntactic constituents (e.g., noun phrase, verb phrases), which are in turn combined into larger structures. Figure 2 presents an example of a constituency parse tree. Although several constituency-based annotation frameworks exist, the Penn Treebank syntactic annotation scheme has been particularly influential in English-language treebanks,[6] including the Penn Treebank (Marcinkiewicz, 1994) and OntoNotes (Weischedel et al., 2022). The Penn Treebank scheme includes labels for major phrase and clause categories, such as declarative clauses (`S`), noun phrases (`NP`), and verb phrases (`VP`). These labels represent the hierarchical organization of syntactic constituents within a sentence.

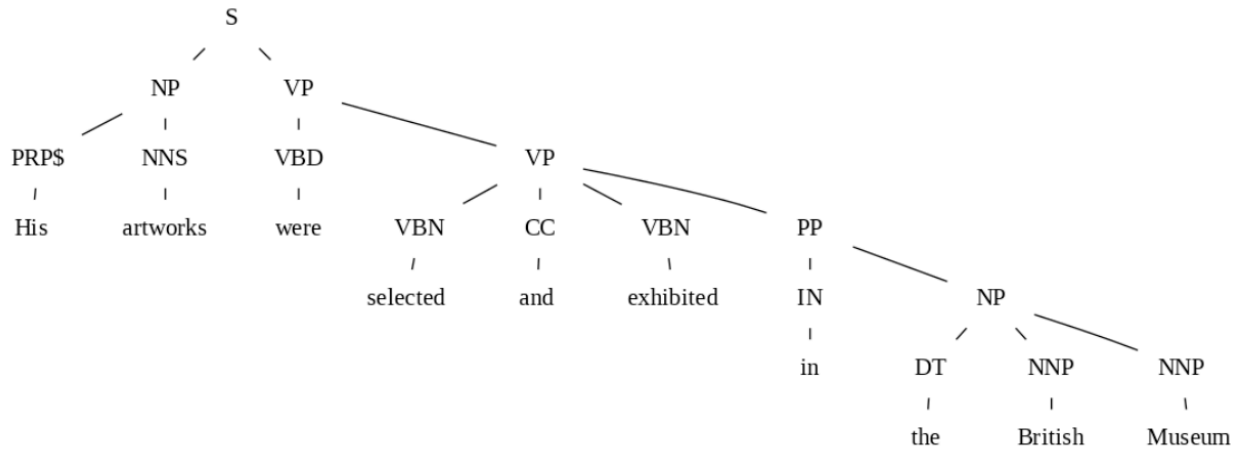


Figure 2: An example of constituency parse tree, visualized using *spaCy* with the *benepar_en3 parser*. Adapted from Kyle and Sung (2026)

(2) A *dependency parse* represents syntactic structure as a set of directed relationships between individual words. Each relationship connects a syntactic *head* to one of its *dependents* and is assigned a label indicating the grammatical function of that relationship. Figure 3 shows an example of a dependency parse.

[6] A *treebank* is a corpus in which sentences have been annotated with syntactic structure.

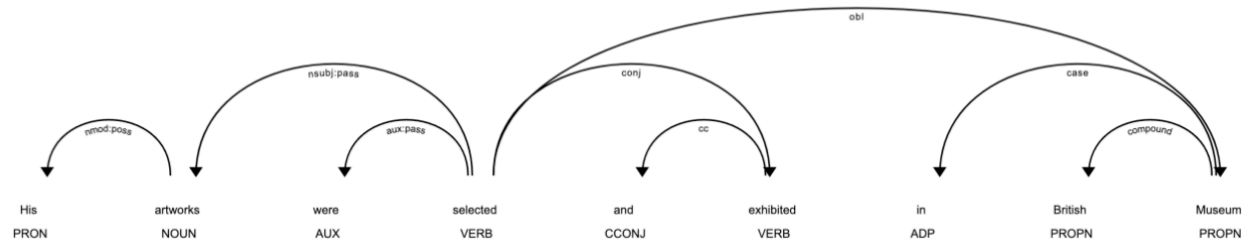


Figure 3: An example of dependency parse tree with Universal Dependencies, visualized by *displaCy*. Adapted from Kyle and Sung (2026)

Some dependency annotation schemes are designed for particular languages, such as CLEARNLP for English (Choi et al., 2015), whereas others are intended to support consistent annotation across many languages. Universal Dependencies (UD), for example, provides a cross-linguistically applicable framework for representing grammatical relationships (Nivre et al., 2020).

Compared with constituency representations, dependency representations explicitly encode grammatical functions between words. For example, UD distinguishes several types of nominal dependents associated with predicates, including nominal subjects (`nsubj`), objects (`obj`), and oblique nominals (`obl`). It also distinguishes relationships between nouns, such as nominal modifiers (`nmod`) and appositional modifiers (`appos`). These dependency labels make it possible to identify not only which words are syntactically related, but also the grammatical function of each relationship.

2.2.2 Challenges in evaluating NLP accuracy

One of the challenges associated with these NLP procedures is that automatic annotation is usually produced through a *pipeline* of interdependent processes. Sentence segmentation may be followed by tokenization, lemmatization, tagging, and parsing, such that errors introduced at an early stage can propagate to subsequent stages and affect the final annotation. However, published accuracy estimates often evaluate individual components in isolation while assuming that all preceding steps have been completed perfectly. For example, POS-tagging accuracy may be calculated using manually corrected token boundaries rather than the automatically generated tokenization that researchers would encounter in actual applications.

NLP systems are commonly evaluated by training an algorithm on approximately 90–95% of a manually annotated corpus and testing it on the remaining 5–10%. During this evaluation, the automatic output must be aligned with the manually annotated reference data. Evaluating a single stage independently is useful for comparing systems and improving individual components, but it can create misleading expectations about performance in a complete learner corpus pipeline.

Overall accuracy may also conceal substantial variation across linguistic features. Frequent or relatively straightforward categories may approach perfect accuracy, while rare or ambiguous

categories remain poorly annotated. A single overall accuracy score therefore does not necessarily indicate whether the specific linguistic feature examined in a study has been annotated reliably.

A further issue is that NLP tools are not necessarily designed to identify the linguistic distinctions of greatest interest to SLA researchers (Meurers and Dickinson, 2017). A general-purpose parser may perform well according to its original evaluation criteria while still requiring additional processing to identify a particular construction or complexity feature. Researchers must therefore evaluate not only the general performance of a tool but also the accuracy of the specific output on which their analysis depends. This evaluation is difficult because manually annotated L2 corpora suitable for training and testing remain limited. Without appropriate reference data, researchers may have little evidence regarding how accurately a tool processes the learner population, genre, proficiency level, or linguistic feature under investigation. The followings are two illustrative studies that evaluated NLP preprocessing pipelines for L2 English (Kyle et al., 2024) and L2 Korean (Sung and Shin, 2025).

Chapter Summary

Learner corpus research uses collected samples of learner language to investigate the characteristics and development of second language use. It can contribute both to SLA research, by describing variation across learners, tasks, proficiency levels, and learning contexts, and to language teaching and assessment. Learner corpus studies may use cross-sectional, longitudinal, or pseudo-longitudinal designs and may focus on lexical, grammatical, phraseological, discourse, pragmatic, or interactional features. NLP supports learner corpus research by automating processes such as tokenization, tagging, and parsing. These processes make it possible to annotate and analyze datasets that may be too large for entirely manual analysis. However, NLP tools are typically developed and evaluated on particular languages, genres, populations, and annotation schemes. Their reported overall accuracy may therefore provide limited evidence about their performance on learner language or on the specific linguistic feature required for a study. Because errors can propagate through an NLP pipeline, researchers should evaluate the accuracy of the particular annotations, categories, or measures on which their conclusions depend.

Activity: Connecting NLP to your RQ

1. Revisit the RQ you are considering for your final project. What would you like to understand about learner language, language development, language teaching, or assessment?
2. What type of learner language data could help you investigate this question? For example, would you examine learner writing, spoken interviews, classroom interaction, assessment responses, discussion forums, or repeated samples collected over time?
3. What research design would be most appropriate: cross-sectional, longitudinal, or pseudo-longitudinal? What conclusions would this design allow you to make, and what conclusions would it not support?
4. What information about the learners, tasks, texts, or learning contexts would be necessary to interpret the corpus appropriately? Consider variables such as first-language background, proficiency, age, instructional context, genre, topic, and task conditions.
5. What linguistic feature or construct would you examine? For example, you might investigate vocabulary, phraseology, grammatical constructions, errors, syntactic complexity, discourse organization, pragmatic functions, or interactional features. How would you define and operationalize this construct?
6. What NLP process or tool could support the analysis?
7. How would you determine whether the NLP output is sufficiently accurate for your research question?
8. What part of the analysis would still require human judgment? Consider tasks such as identifying learner errors, interpreting intended meanings, resolving ambiguous annotations, or evaluating whether a computational measure represents the intended construct.

3 Corpus design and selection

References

- Egbert et al. (2022). Designing and evaluating language corpora: A practical framework for corpus representativeness. Cambridge University Press. [Section 3.1]
- Gries (2009). What is corpus linguistics?. In *Language and linguistics compass*. *3*(5), 1225-1241. [Section 3.2] [Section 3.3] [Section 3.4]

Taking a broader perspective than the preceding chapters, this chapter considers how corpus linguistics has been applied across a wide range of linguistic subfields beyond learner corpus research, including general linguistics, applied linguistics, discourse analysis, sociolinguistics, and computational linguistics. Although the term *corpus* is used in broadly similar ways across these areas, introductory textbooks do not offer a single, universally shared definition. In general terms, *corpus* derives from the Latin word for "body" and refers to a body, or collection, of language data. The diversity of definitions reflects broader differences in how scholars conceptualize both the role of corpora in linguistic research and the nature of corpus linguistics as a field. Some researchers (e.g., Teubert, 2005; Stubbs, 2006) consider corpus linguistics a theoretical approach to the study of language. Others define it primarily as a *methodology* for investigating linguistic research questions (e.g., Gries, 2009). From this methodological perspective, the goal of corpus linguistics is to construct and empirically analyze a sample of texts as the basis for meaningful generalizations about a language or discourse domain. A central concern is therefore whether the corpus adequately represents the domain about which the researcher intends to make claims.

3.1 Sampling and representativeness

The distinction between a *population* and a *sample* is a useful starting point for thinking about corpus design. In statistics, researchers are often interested in a large population that cannot be observed in its entirety. For example, a researcher may want to understand the reading habits of all university students, but it would usually be impossible to ask every university student directly. Instead, the researcher collects data from a smaller sample of students and uses that sample to make careful estimates about the larger population. Corpus design involves a similar logic. The population is the broader domain of language use that the researcher wants to understand (e.g., academic writing, newspaper editorials, classroom interaction, spoken conversation, etc.). The corpus is the sample: the actual collection of texts, transcripts, or language data selected for analysis. Just as a statistical sample should be selected in a principled way, a corpus should be designed so that it can support reasonable generalizations about the language-use domain of interest.

- **Population**: the broader group, domain, or type of language use that the researcher seeks to characterize.
- **Sample**: the actual collection of texts or language data selected from that population.

The population/sample distinction helps clarify why corpus design matters. If a corpus is a sample of a broader language-use domain, then the quality of the corpus depends not only on how much data it contains, but also on how that data was selected and what it is intended to represent. A very large collection of texts may still be limited if the texts were gathered only because they were easy to access. Conversely, a smaller corpus may be useful if it was carefully designed for a specific research question and target domain.

For this reason, corpus design requires researchers to ask several related questions: What broader domain of language use is the corpus supposed to represent? What kinds of texts or speakers should be included? How much data is needed? And what kinds of generalizations can reasonably be made from the resulting sample? These questions are central to how we define a corpus in the first place.

3.1.1 An operational definition of a corpus

A *corpus* can be defined as:

> A large and principled sample of texts designed to represent a target domain of language use, such as a language, dialect, register, genre, or learner population.

This definition highlights three important characteristics: *size*, *principled sampling*, and *representativeness*.

3.1.2 Size.

There is no universally required minimum size for a corpus. Nevertheless, corpus size remains an important consideration because "large" is a relative concept. The amount of data required depends on several factors, including:

- the frequency and distribution of the linguistic feature being investigated;
- the nature and heterogeneity of the domain that the corpus is intended to represent; and
- the goals of the analysis.

A corpus that is sufficiently large for examining frequent lexical items may be too small for investigating rare grammatical constructions or low-frequency discourse features.

3.1.3 Principled sampling.

A corpus should not be viewed simply as a convenient collection of texts. Its design should be guided by explicit criteria concerning which texts, speakers, writers, tasks, genres, or contexts are included. These decisions should be connected to the research questions and the target domain.

3.1.4 Representativeness.

Representativeness concerns the extent to which a corpus can support bettergeneralizations about a broader language-use domain. As discussed earlier, a corpus is not treated as the domain itself, but as a sample drawn from a larger domain of interest. For this reason, claims based on corpus data should be understood as estimates of linguistic patterns in that domain, rather than as direct facts about the domain as a whole.

A useful way to conceptualize representativeness is to distinguish among three levels: the (1) *target domain*, the (2) *operational domain*, and the (3) *corpus*. The (1) target domain refers to the full real-world domain that the researcher ultimately wants to describe. For example, a researcher may be interested in academic writing, newspaper writing, spoken classroom interaction, or argumentative essays written by second language learners. However, such domains are often too broad to observe directly in their entirety. Researchers therefore define an (2) operational domain: a more concrete and researchable version of the target domain, specified in terms of available texts, genres, registers, writers, tasks, time periods, institutions, or other relevant criteria. The (3) corpus is then the actual set of texts selected from this operational domain.

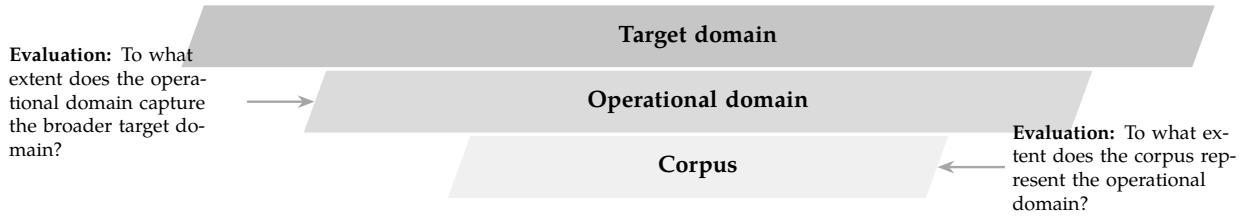


Figure 4: Three levels involved in evaluating corpus representativeness: target domain, operational domain, and corpus. Adapted from Egbert et al. (2022)

As shown in Figure 4, representativeness involves at least two related evaluations. First, researchers must ask whether the operational domain adequately captures the broader target domain. For example, if the target domain is "academic writing," the researcher must decide which disciplines, genres, publication types, institutions, and time periods are included in the operational definition of academic writing. Second, researchers must ask whether the corpus adequately represents the operational domain. This requires attention to how texts were selected, whether important internal categories are included, and whether the sample is large and balanced enough for the intended analysis.

Representativeness should therefore be understood in relative rather than absolute terms. A corpus is not simply representative or unrepresentative in a general sense. Rather, it achieves a degree of representativeness with respect to a particular research goal, a particular domain description, and particular linguistic variables. At the same time, corpus that is appropriate for one research question may be inadequate for another. For example, a corpus designed to estimate the overall frequency of nominalizations in newspaper writing may not be adequate for comparing nominalization use across newspaper sections, historical periods, or political orientations unless

those dimensions were considered in the corpus design.

Evaluating representativeness requires attention to two broad sets of considerations:

- **Domain considerations**: whether the target domain has been clearly described, whether the operational domain is an appropriate approximation of that target domain, and whether relevant internal categories such as genres, registers, speakers, writers, tasks, contexts, institutions, or time periods are adequately represented.
- **Distributional considerations**: whether the corpus is large enough and internally varied enough to provide reliable estimates of the linguistic variables of interest. These variables may include, for example, the average frequency of nominalizations, attributive adjectives, passive constructions, lexical bundles, or particular morpheme types.

3.1.5 Reusing existing corpora.

Representativeness can be considered important when researchers reuse existing corpora. General-purpose corpora can be helpful because they can support many different studies, but their suitability cannot be assumed in advance. A corpus compiled for one purpose may not provide an adequate empirical basis for a different kind of claim. For example, a corpus may be suitable for estimating broad patterns in newspaper writing but unsuitable for comparing political editorials, sports reporting, and financial news if these internal categories were not sampled systematically. Similarly, a learner corpus may support general comparisons between L1 groups but may be less appropriate for analyzing task effects, proficiency differences, or rare grammatical constructions unless those dimensions are adequately represented.

3.1.6 Representativeness and balance

**Note.* This section adds Gries's (2009) discussion of representativeness and balance to the corpus-design framework introduced above.

Many corpora are designed with sampling issues in mind so that researchers can make reasonable generalizations from the corpus to a language as a whole, or to a specific variety, register, or discourse domain. For this reason, corpus compilers often aim to make their corpora both representative and balanced. A corpus is *representative* when it includes the major components of the language variety under investigation. For example, suppose a researcher is interested in phonological reduction in the speech of Californian adolescents. If the corpus only contains conversations among adolescents and their peers, it would not fully represent this variety of speech. This is because adolescents also speak with parents, teachers, and other adults, and these interactions may show different linguistic patterns. Ideally, these contexts should also be included. A corpus is *balanced* when the different components of the target variety are included in appropriate proportions. In other words, the amount of each type of data in the corpus should reflect its relative frequency or importance in the language variety being studied. For example, if dialogs make up approximately 65% of Californian adolescents' speech, then about 65% of the corpus should ideally consist of

dialog recordings.

Again, it is important to note, that representativeness and balance are best understood as theoretical ideals. Corpus compilers use these principles as guiding goals, but there is no single agreed-upon method for creating a corpus that is perfectly representative and perfectly balanced. In practice, corpus design always involves methodological decisions and compromises. For further discussion, Biber's work on corpus representativeness (Biber, 1990, 1993) is particularly useful, and McEnery (2006) also provide helpful discussions of corpus design in Sections A2, A8, and B1.

3.2 Different types of corpora

Corpora can differ in several important ways. These distinctions are useful because different research questions require different types of corpora. For example, a researcher interested in historical language change would need a different corpus from someone studying present-day academic writing or comparing translations across languages.

- General corpora vs. specific corpora
  - A *general* corpus aims to represent a language as a whole.
  - A *specific* corpus is intentionally restricted to a particular language use domain.
- Diachronic corpora vs. synchronic corpora
  - A *diachronic* corpus is designed to represent language change over time.
  - A *synchronic* corpus provides a snapshot of a language or variety at a particular point in time.[7]
- Monolingual corpora vs. parallel corpora
  - A *monolingual* corpus contains texts in one language or language variety.
  - A *parallel* corpus contains the same or comparable texts in multiple languages.
- Static corpora vs. dynamic corpora
  - A *static* corpus has a fixed size. Once it is compiled, no new texts are added.
  - A *dynamic* corpus, or monitor corpus, continues to grow as new materials are added over time.

These categories show that there is no single type of corpus that works for every research question. Choosing an appropriate corpus depends on what kind of language use the researcher wants to investigate: a whole language, a specific register, historical change, translation patterns, or ongoing changes in contemporary usage.

3.3 Different types of information

Corpora contain much more than isolated words. Because corpora represent language as it is used in authentic communicative situations, they include words in larger linguistic and discourse contexts, such as phrases, syntactic patterns, speaker turns, pauses, and intonation patterns in spoken

[7] This "point in time" does not necessarily mean a single year; it may cover a broader period, such as a decade.

data. In addition to the language data itself, corpus files may also include extra information that helps researchers understand, organize, and analyze the data. See Figure 5.

Contextual information

Metadata / Header

```
ID: news_001
Date: 2025-04-12
Source: Daily News
Genre: Editorial
Author: Staff writer
```

Structural information

Markup

```
<title>Should Children Learn Languages Early?</title>
<section>
  <p>Students should begin foreign language learning early... </p>
  <p>However, the quality of instruction is also important... </p>
</section>
```

Language data

Text

Students should begin foreign language learning early because early exposure may support long-term development. However, the quality of instruction is also an important factor.

Interpretive linguistic analysis

Annotation

Token	XPOS	Dependency
Students	NNS	nsubj
should	MD	aux
begin	VB	root
foreign	JJ	amod
language	NN	compound
learning	NN	obj
early	RB	advmod

Figure 5: A simplified example of how metadata, markup, text, and annotation may be stored in or associated with a corpus file.

- Metadata
  - *Metadata* refers to information about the source or context of the corpus data. For example, it may include the publication date and source of a newspaper article, or information about speakers such as age, profession, education level, or number of interlocutors. Metadata may also include copyright or access information.
- Markup
  - *Markup* refers to information about the structure or physical organization of the original document. For example, markup may indicate headings, paragraph breaks, sections, page divisions, or speaker turns. Markup is generally more descriptive and objective than

linguistic annotation.

- Annotation
  - *Annotation* refers to interpretive linguistic information added to the corpus. For example, annotation may include part-of-speech tags, lemmas, syntactic dependencies, semantic categories, discourse functions, or learner error tags. Unlike metadata or markup, annotation reflects a particular linguistic analysis and may therefore be theory-dependent or debatable.

3.4 Different ways to process corpora

Once corpus files have been compiled and organized, researchers need ways to retrieve, transform, annotate, and analyze information from them. The choice of processing method depends on the format of the corpus, the complexity of the research question, and the level of control needed over the analysis. Some research questions can be addressed with ready-made corpus tools, while others require customized processing through programming.

- Corpus analysis programs
  - Many corpus analysis programs provide user-friendly graphical interfaces for common corpus-linguistic tasks.
  - These tools can generate outputs such as concordance lines, frequency lists, keyword lists, collocation tables, and n-gram lists.
  - Examples include *AntConc*, *WordSmith Tools*, and *#LancsBox*.
  - Online corpus platforms such as *Sketch Engine*[8] provide similar functions, often with access to large preloaded corpora and additional tools such as word sketches.
- Programming languages
  - Many corpus linguists use programming languages (e.g., *Python*, R) to process and analyze corpus data.
  - Programming allows researchers to design customized searches, clean and restructure corpus files, extract linguistic features, calculate frequencies, normalize counts, merge metadata, visualize results, and run statistical models.
  - Programming is especially important when the analysis depends on several linked steps (e.g., filtering texts by metadata, extracting annotated features, normalizing frequencies, comparing groups, and evaluating the reliability of the resulting measures).
  - Compared with point-and-click tools, programming requires more initial training, but it provides greater flexibility, transparency, and reproducibility. Because each step can be saved as code, other researchers can inspect, repeat, modify, and extend the analysis.

In practice, these approaches can be combined. A researcher might first use a corpus analysis program to explore concordance lines, then use Python to clean the data, extract features,

[8] https://www.sketchengine.eu/

calculate normalized frequencies, and run statistical analyses. The most important point is that the processing method should match the research question, the corpus format, and the kind of evidence needed to support the study's claims.

A major advantage of programming is flexibility. Ready-made corpus tools are convenient, but they may not handle all file formats, annotation systems, or research needs. They may also make hidden decisions about how the data are processed. For example, a frequency list depends on how the program defines a "word", but this is not always straightforward:

1. Tokenization: As we discussed earlier, this is the process of identifying word tokens in a text, which may involve many decisions. For example, researchers/programs must decide how to treat punctuation, abbreviations, hyphenated words, apostrophes, numbers, spelling variation, and URLs.
2. Headers and annotation: Corpus files may contain headers with metadata. If a program cannot distinguish the header from the main text, it may incorrectly count words in the header as part of the corpus.
3. Customized analysis: Programming allows researchers to define their own search criteria. For example, a researcher may want to retrieve all forms of a verb lemma and save each occurrence with its surrounding context. Such customized tasks may be difficult or impossible in standard concordance programs, but they can be implemented with programming.

In short, graphical tools are useful/accessible, especially for beginners and standard text files. However, programming languages give researchers greater control, flexibility, and transparency. This is especially important when dealing with research questions that require customized processing.

Chapter Summary

Corpus design/selection is a central issue in corpus linguistics because corpus-based claims depend on the relationship between the texts included in a corpus and the broader domain of language use that the researcher wants to understand. A corpus is not simply a convenient collection of texts; it is a sample designed to represent a target language use domain. For this reason, researchers need to consider corpus size, sampling procedures, and representativeness. This chapter also introduced different types of corpora, and showed that corpora contain not only words but also contextual information such as metadata, markup, and annotations, all of which can shape how corpus data are interpreted and analyzed.

Activity: Finding a corpus for your RQ

Goal: Identify a possible corpus for your research and evaluate whether it is appropriate for the claims you want to make.

1. Recall a RQ that you want to investigate using corpus data.
2. Identify the target domain of language use for your question. What broader language-use domain do you want to understand?
3. Search for an existing corpus that might be relevant to your research question. You may search corpus repositories, published studies, institutional corpus projects, or publicly available datasets.
4. Record the basic information about the corpus:
   - corpus name;
   - language or languages included;
   - size;
   - text type, genre, register, or task;
   - speakers or writers represented;
   - time period;
   - available metadata;
   - available annotation;
   - access conditions.
5. Evaluate the corpus in relation to your research question. To what extent does the corpus represent your target domain?
6. Identify possible limitations. For example, are some genres, registers, learner groups, proficiency levels, topics, time periods, or contexts missing or underrepresented?
7. Decide whether the corpus is appropriate for your research question. If it is appropriate, explain what kinds of claims it would support. If it is not appropriate, explain what kind of corpus you would need instead.
8. Share your findings with your group. Compare how different RQs require different corpus designs.

4 Unit of analysis

References

- Webb (2021). The lemma dilemma: How should words be operationalized in research and pedagogy?. In *Studies in Second Language Acquisition*. *43*(5), 941–949. [Section 4.1]
- Jurafsky and Martin (2026). Words and Tokens. In *Speech and Language Processing*. [Section 4.2]

Words are often treated as the basic unit of analysis in NLP. Before counting words in a corpus, however, researchers must determine what constitutes a word. This involves two related but distinct decisions. First, they must select a theoretically appropriate *lexical unit*, such as a word type, lemma, flemma, or word family. Second, they must determine how that unit will be identified computationally in running text (i.e., *tokenization*). These decisions should not be conflated. A tokenizer segments text into computational units, but the resulting tokens do not necessarily correspond directly to the lexical units required for a particular corpus analysis.

4.1 Lexical units

Words can be defined and categorized in several ways. One important distinction concerns how different morphological forms are grouped together. Common units include *word types*, *lemmas*, *flemmas*, and *word families*, as described in Table 1.

Table 1: Lexical units

Unit	Definition	Example
Word type	Each distinct word form is treated as a separate unit.	*record*, *records*, *recorded*, and *unrecorded* are different word types.
Lemma	A headword and its inflectional forms belonging to the same part of speech.	The noun forms *record* and *records* constitute one lemma. The verb forms *record*, *records*, *recorded*, and *recording* constitute a separate lemma.
Flemma	A headword and its inflectional forms, without distinguishing between parts of speech.	The noun and verb forms *record*, *records*, *recorded*, and *recording* are grouped into one flemma.
Word family	A headword together with its inflectional and derivationally related forms.	A word family may include *record*, *records*, *recorded*, *recording*, *recorder*, *pre-record*, and *unrecorded*.

The choice of lexical unit affects estimates of vocabulary size, lexical diversity (e.g., Jarvis and Hashimoto, 2021), lexical coverage, and vocabulary knowledge. It is therefore not merely a technical preprocessing decision. It is part of the operationalization of the research construct.

4.1.1 Larger units in vocabulary pedagogy

The greatest value of larger lexical units may lie in pedagogy. Presenting headwords together with their inflectional and derivational forms may provide a shortcut to lexical development. Learning several related forms may be easier than learning the same number of morphologically unrelated words. For example, once a learner knows *record*, knowledge of English morphology may support the comprehension or learning of forms such as *recorded*, *recording*, and *recorder*. Organizing vocabulary around lemmas or word families can therefore highlight systematic relationships between words. However, knowledge of a headword does not necessarily imply complete knowledge of all its related forms. Learners may recognize a relationship between two forms without being able to understand or produce both forms accurately.

4.1.2 Smaller units in vocabulary research

Smaller units may often be more useful in research because they require fewer assumptions about learners' morphological knowledge. Treating forms separately allows researchers to distinguish knowledge of a base word from knowledge of its inflections and derivations. This does not mean that a smaller unit is always preferable. The most appropriate unit depends on the research question and the population being studied. Several factors should be considered.

Learner proficiency. For less proficient learners, smaller units may be more defensible because researchers cannot assume that knowledge of one form extends to related forms. A learner who knows *decide*, for example, may not necessarily know *decision*, *decisive*, or *indecisive*. Larger units may become more plausible for more proficient learners who have developed greater knowledge of the inflectional and derivational systems of the target language. Nevertheless, even advanced learners may not know all members of a word family.

Receptive and productive knowledge. The appropriate lexical unit may also depend on whether receptive or productive knowledge is being investigated. A learner may recognize the meaning of a derivative while reading but remain unable to produce it accurately. Similarly, being able to use one member of a word family does not ensure that other members can be used appropriately.

Research suggests that both first- and second-language learners may find it difficult to produce all of the derivatives associated with a known headword (e.g., Nation, 2016; Nation and Webb, 2011; Vilkaitė-Lozdienė and Schmitt, 2020). Word families may therefore be particularly problematic as units for measuring productive vocabulary knowledge.

4.1.3 Effects on vocabulary research and pedagogy

The choice of lexical unit has implications for several areas of vocabulary research and pedagogy.

Vocabulary learning. Research on L2 vocabulary learning may ask:

- To what extent does the choice of lexical unit influence estimates of vocabulary learning?
- To what extent are inflected and derived forms of known words recognized when they are encountered in meaningful contexts?
- Does knowledge of a base form facilitate the comprehension or learning of related forms?
- How do these relationships vary according to proficiency, morphological knowledge, frequency, and transparency?

One possible advantage of using larger lexical units is greater ecological validity (the same rationale we discussed earlier for using corpus data to study language as it is actually used.).In natural language use, learners encounter words in different inflected and derived forms rather than only as isolated dictionary headwords. A larger unit may therefore better represent the vocabulary demands of authentic reading or listening. The corresponding disadvantage is a lack of precision. If a learner understands *unpredictable*, a word-family analysis may treat this as knowledge of the PREDICT family. The analysis does not reveal whether the learner already knew *predict*, inferred the meaning from its morphemes, or learned *unpredictable* as a separate lexical item.

Word lists. Different word lists use different lexical units. Some lists contain word types, whereas others are organized around lemmas, flemmas, or word families. The lexical unit affects:

- the reported size of the list
- the number of forms represented by each entry
- the treatment of inflections and derivations
- estimates of lexical coverage
- comparisons between lists

A list of 2,000 word families represents considerably more surface forms than a list of 2,000 word types. These figures are therefore not directly comparable unless the unit of counting is considered.

Vocabulary assessment. One advantage of using larger lexical units in vocabulary tests is that the test can sample morphologically unrelated headwords. Testing *record*, *recorded*, and *recording* as three separate vocabulary items may partly measure knowledge of English morphology rather than knowledge of three distinct lexical meanings. Sampling one form from each word family may therefore provide an efficient estimate of the number of distinct lexical bases that a learner knows.

Lexical coverage and profiling. Lexical coverage refers to the percentage of running words in a text that are accounted for by vocabulary known to a learner. In general, the greater the proportion of known vocabulary in L2 input, the more likely the input is to be understood.

Coverage estimates depend on the lexical unit used. Consider the following forms:

develop, develops, developed, development, developmental

A word-type analysis treats these as five separate units. A lemma-based analysis groups some of the inflected forms but separates forms belonging to different parts of speech. A word-family analysis may treat all five as one unit. The same text may therefore produce different coverage estimates depending on whether the analysis uses word types, lemmas, flemmas, or word families. If a learner knows a headword but does not understand one of its derivatives, counting the entire family as known may overestimate effective lexical coverage.

4.2 Tokenization

Once researchers have selected an appropriate lexical unit, they must determine how units will be identified in corpora. This requires *tokenization*, which is generally the first stage of modern NLP processing. Tokenization refers to the process of segmenting continuous input into a sequence of units called *tokens*. For example, a *tokenizer* might represent the sentence

The students recorded the interview.

as

```
The | students | recorded | the | interview | .
```

In this representation, punctuation is treated as a separate token. The resulting tokens can subsequently be counted or used as input for POS tagging, lemmatization, parsing, and other forms of linguistic analysis.

4.2.1 Tokens and types

A *token* is an individual occurrence of a unit in a text, whereas a *type* is a distinct form among those tokens. Consider the sentence:

The student recorded the interview.

If capitalization is ignored and punctuation is counted, the sentence contains six tokens:

```
the | student | recorded | the | interview | .
```

However, it contains only five types because *the* occurs twice. The number of running tokens is conventionally represented as N, while the number of distinct types is represented as $|V|$, where V is the vocabulary of the text.

These counts depend on tokenization decisions. For example, researchers must decide whether punctuation is counted, whether a contraction such as *I'm* is treated as one token or two, and how numbers, abbreviations, and hyphenated expressions are handled. Note that, as with the operationalization of lexical units discussed in the previous section, there is no single tokenization scheme that is appropriate for every study. Tokenization decisions should instead be guided by

the language, the type of data, and the purpose of the analysis.

4.2.2 Written and spoken Language

In *written* English, spaces and punctuation provide useful indications of possible word boundaries. However, orthographic forms do not always correspond to linguistic words. For example, *I'm* may be treated as one orthographic unit or divided into the pronoun *I* and the contracted verb *'m*. *Spoken* language presents additional difficulties. Transcripts may include filled pauses such as *um* and *uh*, repetitions, false starts, word fragments, laughter, and overlapping speech. Whether these elements should be treated as words or tokens depends on the intended application. A filled pause may be excluded from a vocabulary analysis but retained in a study of fluency or interaction.

4.2.3 Word boundaries across languages

Space-based tokenization does not provide a universal definition of a word. Languages differ considerably in how word boundaries are represented in writing. For example, in *Chinese*, spaces do not normally mark word boundaries. Written forms consist of characters, and each character often represents a morpheme. However, a Chinese word may consist of more than one character, meaning that neither whitespace segmentation nor character segmentation directly identifies words. *Japanese* and *Thai* also do not consistently use spaces to mark word boundaries. At the same time, individual characters are generally too small to serve as appropriate word-level units. *Korean* uses spaces, but spaces mark boundaries between *eojeol* units rather than necessarily between individual lexical words or morphemes. An eojeol may contain a lexical stem together with one or more grammatical morphemes. For example, a noun may occur with a case particle, and a verb stem may occur with tense or sentence-ending morphology.

4.2.4 Morphemes as possible units

A morpheme is a meaningful subpart of a word. For example, the comparative form *longer* can be analyzed as the root *long* and the morpheme *-er*. Morphemes can be broadly divided into roots and affixes. The root carries the central lexical meaning, while affixes add grammatical or lexical information.

- *Inflectional* morphemes express grammatical information such as tense, number, case, or agreement. They are often productive, their meanings are relatively predictable, and they may be required in particular grammatical contexts.
- *Derivational* morphemes form new lexical items or change the meaning or grammatical category of a base. Their distribution and interpretation are often less predictable.
- *Clitics* behave syntactically like words but are phonologically, and sometimes orthographically, attached to another form.

Morphemes might appear to provide a useful alternative to words, but languages differ in their morphological organization. Some languages tend to express grammatical information

through separate words, while others combine several morphemes within a single word.[9] Languages also differ in how clearly morpheme boundaries can be identified.[10]

4.2.5 Characters and Unicode

Characters provide a more clearly defined computational unit than words or morphemes. Before a computer can process written text, however, the characters must be represented in a form that the computer can store and interpret. Computers ultimately store information as numbers, so a shared system is needed to specify which number corresponds to each character. This system is called *character encoding*. Without a consistent encoding system, the same numerical value could be interpreted as different characters by different programs, or a character could be displayed incorrectly. This is important for multilingual text, since writing systems contain far more characters than can be represented by older encoding systems designed primarily for English.

Unicode provides a standardized inventory in which each character is assigned a unique numerical identifier, called a *code point*. UTF-8 is a widely used encoding that specifies how these Unicode code points are stored as bytes in a computer.

For most corpus research, the main practical consideration is that texts should use a consistent encoding, typically UTF-8. Inconsistent or incorrect encoding can cause characters to be displayed incorrectly or treated as different units during searching, tokenization, and frequency counting. However, character-level tokenization does not solve the linguistic problem entirely. Characters are relatively easy to identify computationally, but they are often too small to serve as useful units for representing words and meanings.

4.2.6 Subword tokenizers

Words and morphemes appear to be useful processing units because they often have relatively stable form–meaning associations. However, they are difficult to define consistently across languages. A further challenge is that the number of distinct words continues to grow as more text is observed. Consequently, a word-based NLP system will regularly encounter forms that were not included in its training data.

Modern NLP systems commonly address these problems through *subword tokenization*. A subword token may correspond to a complete word, a meaningful part of a word, or a smaller sequence of characters. Because these units can be recombined, a tokenizer can represent words that were not encountered as complete forms during training. For example, suppose a system has encountered *low*, *new*, and *newer*, but has not encountered *lower*. A strictly word-based system may treat *lower* as unknown. A subword tokenizer may instead represent it as:

```
low | er
```

[9] This distinction is commonly described as a continuum from more *analytic* languages to more *synthetic* languages.

[10] In more *agglutinative* systems, morphemes tend to remain relatively distinct and segmentable, whereas in more *fusional* systems, a single form may express several grammatical meanings simultaneously.

The system can therefore process the unfamiliar word using smaller units that are already part of its vocabulary.

Byte-Pair Encoding. Byte-pair encoding (BPE) is a commonly used method for learning a vocabulary of subword tokens. It begins with small units, typically individual characters or bytes, and repeatedly merges the adjacent pair that occurs most frequently in the training data. As the process continues, frequently occurring character sequences become single tokens, whereas rare words remain divided into smaller units. For example, suppose the training data contains many occurrences of words such as *low*, *lower*, and *lowest*. The tokenizer may first merge `l` and `o` into `lo`, and then merge `lo` and `w` into `low`. If `e` and `r` also occur frequently together, they may be merged into the token `er`. The word *lower* could then be represented as:

```
low | er
```

However, this segmentation is learned from frequency patterns rather than from morphological knowledge. BPE does not identify *low* as a root or *-er* as a suffix; it simply learns that these character sequences occur frequently enough to be included in its token vocabulary.

Chapter Summary

Words are often treated as basic units of corpus analysis, but what counts as a word depends on both theoretical and computational decisions. Researchers must first select an appropriate *lexical unit*. The choice of lexical unit affects estimates of downstream tasks. Researchers also determine how units will be identified in running text. *Tokenization* divides text into computational units called tokens, but there is no universal tokenization scheme. Words and morphemes provide potentially meaningful processing units, but they are difficult to define consistently across languages. Modern NLP systems therefore commonly rely on subword tokenization, which takes a more bottom-up, data-driven approach to segmenting text into processing units. In short, the central distinction is between *lexical-unit selection* and *tokenization*. Lexical-unit selection concerns which forms should be counted separately or grouped together in order to operationalize the linguistic construct being investigated. Tokenization concerns how running text is segmented into units that can be processed computationally.

5 Word vectors

References

- Lenci (2018). Distributional Models of Word Meanings. *Annual Review of Linguistics*. [Section 5.1]
- Jurafsky and Martin (2026). Embeddings. In *Speech and Language Processing*. [Section 5.2]

Just as biological organisms are shaped by their environments, words are shaped by the contexts in which they occur. In this sense, context is essential for understanding the *word*. Words that occur in similar contexts tend to have similar meanings. This idea is generally referred to as the distributional hypothesis.

5.1 Distributional semantics

Before examining how word meanings can be represented computationally as vectors, we first need to consider a more fundamental question: How can a computer represent a word and its meaning? At the simplest level, a word can be represented as a string of characters, such as `dog`, or as an index in a vocabulary. For example, if a vocabulary contains three words, they might be represented as follows:

$$dog \rightarrow 1 \qquad cat \rightarrow 2 \qquad table \rightarrow 3$$

Such representations allow a computer to distinguish one word from another, but the identifiers themselves contain no information about meaning. There is nothing about the numbers 1 and 2, for example, that tells us that *dog* and *cat* are more semantically similar than *dog* and *table*.

One solution is to represent lexical meaning through explicitly encoded semantic information. Lexical resources such as *WordNet*[11], for example, organize words according to semantic categories and relations. Another approach, however, is to infer semantic information from patterns of language use itself. This is the basic idea behind *distributional semantics*.

Distributional semantics represents linguistic items on the basis of the contexts in which they occur. Its central assumption (i.e., *distributional hypothesis*) is that lexical items occurring in similar linguistic contexts tend to have similar meanings. For example, *dog* and *cat* may both frequently occur with words such as *pet*, *feed*, *animal*, and *vet*. In contrast, a word such as *car* is more likely to occur with words such as *drive*, *road*, and *engine*. From these differences in distribution, we can infer that *dog* is semantically more similar to *cat* than to *car*.

The next question is how such distributional information can be represented computationally. Distributional models typically represent each lexical item as a numerical *vector*. A vector summarizes how a target word is distributed across a set of contexts.

For example, suppose that we define four context words—*bark*, *pet*, *food*, and *engine*—and

[11] https://wordnet.princeton.edu/

count how frequently each target word occurs with them:

Target word	*bark*	*pet*	*food*	*engine*
dog	120	95	60	1
cat	45	88	55	0
car	0	2	1	130

Each row can then be represented as a vector:

$$\vec{dog} = [120, 95, 60, 1]$$

$$\vec{cat} = [45, 88, 55, 0]$$

$$\vec{car} = [0, 2, 1, 130].$$

The *dog* and *cat* vectors have relatively similar patterns. Both have high values for contexts such as *pet* and *food* and very low values for *engine*. The *car* vector, in contrast, has a very different distribution. A distributional model quantifies the similarity between these vectors and use this similarity as an approximation of semantic similarity.

The resulting vector depends on what we mean by *context*. Context can be defined in several different ways. One approach is to use words occurring within a fixed window around the target word. For example, in "The small dog chased the cat," a window-based model might use nearby words such as *small*, *chased*, and *cat* as contextual information for *dog*. This approach captures patterns of linear co-occurrence without explicitly representing grammatical relationships. A second possibility is to define contexts in terms of syntactic relations. In the example sentence above, a dependency-based representation might encode *dog* as the subject of *chase* and *cat* as its object. This allows the model to distinguish syntactic relationships that may be obscured in a simple linear window (i.e., a sentence without any layer of syntactic structures). Because these relations must first be identified computationally, such models generally require dependency-parsed corpora. A broader alternative is to treat an entire text or document as a context. For example, if *inflation* and *interest rate* frequently occur in the same news articles, their distributions across documents may provide evidence that the two expressions are semantically related.

Thus, distributional models share the general assumption that semantic information can be inferred from patterns of linguistic use, but they may differ substantially in how those patterns are defined and represented. The choice of context (whether neighboring words, syntactic relations, or larger textual units) plays an important role in determining what kinds of semantic similarity a distributional representation captures.

5.2 Vector semantics

In modern NLP, *vector semantics* is the common/standard approach to representing word meaning. It models semantic relationships among words by representing them as points, or *vectors*, in a multidimensional space, where their relative positions reflect aspects of their meaning.

The idea of representing meaning spatially has a long history. Osgood et al. (1957), for example, represented the connotative meaning of words as locations in a multidimensional semantic space. From a different perspective, Firth (1957) emphasized the importance of linguistic context in understanding word meaning, famously stating that "*you shall know a word by the company it keeps*" (p. 11). Modern vector semantics brings these ideas together: meaning is represented spatially, while the position of a word in that space is derived from patterns of language use.

A word is represented as a point in a multidimensional semantic space, and that position is derived from the distributions of words that occur around it. Consider *dog* and *cat* again. If these words frequently occur with many of the same neighboring words, their numerical representations should reflect those similar distributions. They should therefore be located relatively close together in semantic space. By contrast, if *dog* and *car* occur in very different linguistic environments, their representations should be farther apart. In this way, distributional patterns observed in language are transformed into numerical representations whose spatial relationships encode aspects of semantic similarity. These numerical representations are commonly called *embeddings*. The term *embedding* refers to the mathematical idea of mapping objects into a continuous vector space. In NLP, words or other linguistic units are mapped to vectors so that relationships among them can be represented geometrically, for example through the distance or direction between points in the space.[12]

5.2.1 Simple count-based embeddings

A simple way to construct a vector representation is to count how often words occur together. This can be represented using a *co-occurrence matrix*. Suppose we record how often three target words occur with several neighboring words:

Table 2: A simplified co-occurrence matrix

	pet	*animal*	*bark*	*meow*
dog	20	15	18	1
cat	19	16	1	17
democracy	0	0	0	0

Each row can be treated as a vector representing a word. For example, the vector for *dog* is:

$$\mathbf{dog} = [20, 15, 18, 1]$$

[12] One way to explore such representations visually: https://projector.tensorflow.org/.

and the vector for *cat* is:

$$\mathbf{cat} = [19, 16, 1, 17].$$

These vectors summarize the distributions of the words across different linguistic contexts. A *vector space* is a collection of vectors and is characterized by its number of dimensions. In the simplified example above, each vector has four dimensions because there are four context words. In an actual corpus, however, we may compute word vectors using all words in the vocabulary as possible contextual dimensions. If the vocabulary contains $|V|$ words, the resulting vectors may therefore have $|V|$ dimensions. For example, if the vocabulary contains 50,000 words, a word may conceptually be represented as a point in a 50,000-dimensional space. We compute these word vectors over an entire corpus by recording patterns of co-occurrence. However, any individual word will co-occur with only a subset of all words in the vocabulary. As a result, most entries in the vector will be zero. A vector might therefore look conceptually like:

$$[0, 0, 3, 0, 0, 12, 0, 0, 5, 0, \ldots]$$

This is called a *sparse* vector representation. The vector is very long, but most of its values are zero. Efficient algorithms exist for storing and computing sparse matrices without having to treat every zero as an independently stored value. The important point is that each word can now be represented as a numerical point in a $|V|$-dimensional vector space, and its position in that space reflects its distribution across linguistic contexts.

5.2.2 Cosine for measuring similarity

Once two words have been represented as vectors, we need a way to quantify how similar those vectors are. Suppose two target words are represented by vectors $\mathbf{v}$ and $\mathbf{w}$. Because both vectors are constructed using the same contextual dimensions, they have the same dimensionality and can therefore be compared directly.

The most common similarity measure is the *cosine of the angle between two vectors*, usually referred to as *cosine similarity*. The basic intuition is that two vectors pointing in similar directions represent similar patterns across their dimensions. Thus, if two words tend to occur across contexts in similar proportions, their vectors should point in similar directions, even if one word is much more frequent overall than the other.

Cosine similarity is calculated as:

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\|\|\mathbf{w}\|}$$

The numerator, $\mathbf{v} \cdot \mathbf{w}$, is the dot product of the two vectors. The denominator divides this value by the lengths, or *magnitudes*, of the two vectors. This normalization makes cosine similarity depend primarily on the direction of the vectors rather than their absolute size, because dividing by

each vector's magnitude removes differences that are due simply to one vector being larger than the other. For example, suppose two words are represented across three contextual dimensions:

$$\mathbf{v} = [10, 20, 30], \qquad \mathbf{w} = [1, 2, 3]$$

Although the first word occurs ten times more frequently in every context, the relative distribution across the three dimensions is identical. Their cosine similarity is:

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{10 \cdot 1 + 20 \cdot 2 + 30 \cdot 3}{\sqrt{10^2 + 20^2 + 30^2}\,\sqrt{1^2 + 2^2 + 3^2}}$$

$$= \frac{10 + 40 + 90}{\sqrt{1400}\sqrt{14}} = \frac{140}{140} = 1$$

Thus, even though $\mathbf{v}$ is much larger in magnitude than $\mathbf{w}$, the two vectors point in exactly the same direction because their values occur in the same proportions across all dimensions.[13]

5.2.3 Word2Vec

Simple count-based embeddings provide an intuitive starting point for turning words into numerical representations. However, they also have an important limitation. If each word in the vocabulary corresponds to one dimension, the resulting representation is both long and sparse. A vocabulary containing tens of thousands of words produces vectors containing tens of thousands of dimensions, most of which may be zero. A more powerful alternative is to learn shorter, *dense* embeddings. Instead of using the vocabulary size $|V|$ as the number of dimensions, dense embeddings typically use a much smaller number of dimensions d, often ranging from approximately 50 to 1000. Thus, instead of a representation such as:

$$[0, 0, 3, 0, 0, 12, 0, 0, 5, 0, \ldots]$$

with a very large number of mostly zero dimensions, a dense embedding might look conceptually like:

$$[0.21, -0.43, 0.72, 0.08, \ldots].$$

The values in dense embeddings are real-valued numbers and can be positive or negative. Unlike the dimensions in a simple co-occurrence matrix, the individual d dimensions generally do not have clear interpretations. We cannot ordinarily say, for example, that Dimension 1 corresponds to one

[13] Cosine similarity ranges theoretically from -1 to 1. A value of 1 indicates that two vectors point in the same direction, 0 indicates that they are orthogonal and share no directional similarity, and -1 indicates that they point in exactly opposite directions. In traditional count-based distributional models, where vector values are typically non-negative, cosine similarities usually fall between 0 and 1. The mathematical details of cosine similarity and vector normalization are discussed in greater depth in Jurafsky and Martin (2026), Embeddings, Section 5.2. An interactive visualization of vector angles and cosine similarity is available at: https://editor.p5js.org/_annalena/full/FQmm8akQj.

specific context word or semantic property. Instead, the representation of meaning is distributed across the learned dimensions.

Table 3: Count-based vectors vs. Dense embeddings

	Count-based vector	**Dense embedding**
Dimensionality	Typically related to vocabulary size $\|V\|$	A smaller number of dimensions d, often 50–1000
Representation	Sparse	Dense
Values	Mostly zero counts or functions of counts	Real-valued numbers that may be positive or negative
Dimensions	Correspond to words or contexts	Do not usually have a clear individual interpretation

Skip-gram with negative sampling. One method for computing dense embeddings is skip-gram with negative sampling. Skip-gram is one of two algorithms associated with the *Word2Vec* software package developed by Mikolov et al. (2013b,a). The intuition behind skip-gram with negative sampling is different from directly counting co-occurrences. Suppose w is a target word and c is a possible context word. A count-based approach asks: How often does context word c occur near target word w? Skip-gram instead turns this into a binary prediction problem: Is word c likely to occur near word w? For example, suppose the corpus contains:

The *dog* barked loudly.

If *dog* is the target word, *barked* is a neighboring context word. The pair can therefore be treated as an example of two words that actually occurred near one another. By contrast, we can randomly choose another word from the vocabulary that did not occur in this context: *dog – democracy*. We can then train a classifier to distinguish between the two kinds of pairs. Importantly, the binary classification task itself is not what we ultimately care about. The purpose of the task is to learn numerical parameters that capture useful relationships among words. Once training is complete, we take the learned weights as the word embeddings. This approach is an example of *self-supervision*. We do not need humans to manually label word pairs as semantically similar or dissimilar. Instead, running text provides the training signal: words that actually occur near one another provide positive training examples, while other words can be sampled to create negative examples.

Word2Vec as a prediction-based embedding model. Word2Vec is a prediction-based method for learning static word embeddings. Unlike the neural language models discussed later, however, it uses a much simpler prediction objective and model architecture. In skip-gram with negative sampling, the prediction task is formulated as *binary classification*. Given a target word and a context word, the model predicts whether the pair is a genuine co-occurrence observed in the corpus or a randomly generated negative example. In simple terms, the model asks: Is this target–context pair likely to have occurred together in the corpus? This differs from a neural language model, whose

task is typically to predict a word from its surrounding context or preceding words. Word2Vec also uses a relatively simple model for this classification task.

Importantly, the binary predictions themselves are not usually the final goal. Instead, the prediction task serves as a *training objective*: in learning to distinguish genuine word–context pairs from negative examples, the model adjusts its parameters so that words occurring in similar contexts acquire similar vector representations. The resulting vectors are the word embeddings that we ultimately want to use. If you want to take a closer look into the process of learning Word2Vec embeddings (with the skip-gram with negative sampling example), the next section provides a summary.

Learning Word2Vec embeddings.

- Step 1: Create positive examples. First, take a target word and one of its neighboring context words. For example, in: "The dog barked loudly," we might treat the pair:

  $$(dog, barked)$$

  as a *positive example*. Here, *positive* does not refer to positive meaning or sentiment. It simply means that this target word and context word actually occurred near one another in the corpus. More generally, a positive pair can be represented as:

  $$(w, c_{pos}).$$

- Step 2: Create negative samples. Next, randomly sample other words from the vocabulary to create *negative samples*. For example:

  $$(dog, democracy)$$

  might be used as a negative pair. More generally:

  $$(w, c_{neg})$$

  represents a target word paired with a negatively sampled context word.
- Step 3: Train a classifier. The positive and negative pairs are used to train a logistic regression classifier that distinguishes the two cases. At the beginning of training, the model has an initial set of embeddings. The learning algorithm then adjusts those embeddings based on the positive and negative training examples. The goal is to:
  1. maximize the similarity between the target word and context word in positive pairs,

     $$(w, c_{pos});$$

2. minimize the similarity between the target word and negatively sampled context words,

$$(w, c_{neg}).$$

Consider the following simplified training examples:

Target	Context	Training example
dog	*bark*	Positive
dog	*pet*	Positive
dog	*democracy*	Negative
dog	*equation*	Negative

During training, the model adjusts the embeddings so that the representations involved in positive pairs become more similar, while those involved in negative pairs become less similar. Because many different words appear in many different positive and negative contexts throughout the corpus, repeated training gradually produces vectors that encode broader distributional patterns.

- Step 4: Retain the learned word vectors as embeddings. During training, the model repeatedly updates the vector values associated with each word so that genuine target–context pairs receive higher scores than negative pairs. Once training is complete, the binary classification task is no longer needed. The learned word vectors themselves are retained as the final *word embeddings*. The final embeddings are therefore not manually defined semantic features. They are numerical representations learned by repeatedly adjusting the vectors so that observed target–context relationships receive greater similarity and negative samples receive lower similarity.

Chapter Summary

Distributional semantics is based on the idea that words occurring in similar linguistic contexts tend to have similar meanings. Vector semantics operationalizes this idea by representing words as numerical vectors in a multidimensional space. The position of a word in this space reflects its distribution across linguistic contexts, allowing semantic relationships among words to be represented geometrically. A simple way to construct word vectors is to count how often words occur with different contextual features. These count-based representations are typically high-dimensional and sparse because each contextual feature may correspond to a separate dimension. Modern embedding methods often learn shorter, dense representations. For example, *Word2Vec* learns static word embeddings through a prediction-based training objective.

Activity: Applying word embeddings to corpus linguistic research

1. What kinds of words or lexical categories could be investigated using embeddings?
2. What kinds of corpus comparisons could be made? For example, across registers, time periods, learner groups, or languages.
3. What kinds of corpus-based research questions could these comparisons help answer?
4. What limitations should be considered when interpreting embedding-based results?

6 Neural networks

References

- Jurafsky and Martin (2026). Neural Networks. In *Speech and Language Processing*.

Neural networks are one of the central computational tools in modern natural language processing. Although they are now strongly associated with recent advances in deep learning, the basic idea of a neural network is not new. Early work was partly inspired by biological neurons, in the sense that complex behavior was modeled as emerging from many simple processing units connected together. In modern NLP, however, neural networks are best understood not as literal models of the brain, but as flexible computational architectures composed of many small units that transform numerical inputs into numerical outputs.

A neural network can be viewed as a system for learning useful representations directly from data. In earlier statistical approaches such as logistic regression, much of the work involved designing informative feature templates by hand. For example, for a classification task, a researcher might manually specify features based on surrounding words, suffixes, capitalization, or other patterns that domain knowledge suggested would be useful. Neural networks reduce the need for such extensive hand-engineering. Instead, they can often take relatively raw inputs (such as token indices or embeddings) and learn internal features automatically as part of the training process. This shift from manual feature design to representation learning is one of the key reasons neural networks have become important in NLP.

6.1 A single neural unit

The fundamental building block of a neural network is a *unit* (also called a *node* or *neuron*). A unit takes a set of real-valued inputs, performs a computation on them, and produces a single output. Suppose a unit receives inputs $x_1, x_2, \ldots, x_n$. Each input is associated with a corresponding weight $w_1, w_2, \ldots, w_n$. Intuitively, the weights determine how strongly each input contributes to the unit's computation. A large positive weight increases the influence of the corresponding input, a negative weight reverses its effect, and a weight near zero minimizes its contribution. In addition to these weighted inputs, a neural unit also includes a *bias* term, usually written as b. The bias allows the unit to shift its decision boundary independently of the input values. One common way to conceptualize the bias is as a weight attached to an extra input that is always fixed at $+1$. This is why Figure 6 shows the bias as an additional gray input line.

The first step in the computation is to calculate the weighted sum of the inputs:

$$z = b + \sum_{i=1}^{n} w_i x_i$$

This value z is sometimes called the *pre-activation* or *net input*. It is simply a linear combi-

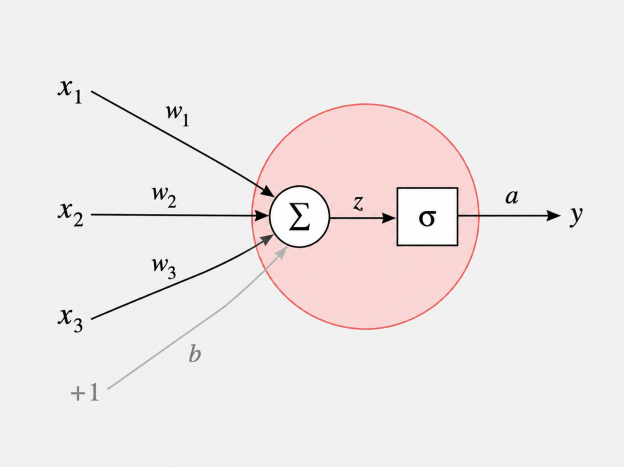


Figure 6: Adapted from Figure 6.2 in Jurafsky and Martin (2026), Chapter 6, p. 3

nation of the inputs. For convenience, this same computation is often written in vector notation. If we let $\mathbf{x}$ be the input vector and $\mathbf{w}$ be the weight vector, then the weighted sum can be written as:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

This notation is compact and useful because neural network computations are typically implemented as vector and matrix operations.

6.2 From weighted sum to activation

If a neural unit stopped at the weighted sum z, it would behave like a *linear* model. However, neural networks become powerful because they apply a *nonlinear* activation function to this weighted sum. The output of the activation function is called the *activation* of the unit, written here as a:

$$y = a = f(z)$$

In Figure 6, the activation function is represented by the box labeled σ, and the activation value a is also the final output y of the unit. In deeper networks, it is often useful to distinguish between the activation of an individual unit and the final output of the entire network, but in a single-unit example these are the same.[14]

[14]The activation function is crucial because it introduces nonlinearity. Without nonlinearity, even a network with many layers would collapse into the equivalent of a single linear transformation. In short, nonlinear activation functions allow neural networks to capture more complex patterns in data.

6.3 Common activation functions

Different neural network architectures use different activation functions. Three of the most common are the (1) sigmoid function, the (2) hyperbolic tangent (*tanh*), and the (3) rectified linear unit (*ReLU*). The *sigmoid* function maps any real-valued input onto a value between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This function is useful when we want to interpret the output as a *probability*. For example, logistic regression uses the sigmoid function to map a linear score to a probability of class membership. Historically, sigmoid activations were also widely used in hidden layers of neural networks. However, because they can saturate for large positive or negative inputs, they may lead to optimization difficulties in deep networks.

The *tanh* function is closely related to the sigmoid, but maps inputs to the range $[-1, 1]$ rather than $[0, 1]$:

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Like sigmoid, $\tanh$ is nonlinear and smooth, but because it is centered around zero, it is sometimes easier to optimize in practice. It was commonly used in recurrent neural networks and older feedforward models.

The *ReLU* (rectified linear unit) has become one of the most widely used activation functions in modern deep learning:

$$\text{ReLU}(z) = \max(0, z)$$

ReLU outputs 0 when the input is negative and simply returns the input itself when it is positive. This simple behavior makes it computationally efficient and often easier to train in deep networks. For this reason, ReLU and its variants are now standard choices in many hidden layers.

For most corpus linguists who use NLP models as analytical tools, the choice of activation function is rarely something that needs to be tuned directly. Activation functions are typically part of the model architecture, whereas the optimizer adjusts learnable parameters such as weights and biases during training. Moreover, when researchers use pretrained NLP models, these architectural choices have already been made by the model developers. Nevertheless, understanding activation functions is important because they help explain how neural networks differ from simpler linear models and how they are able to learn complex, nonlinear patterns in linguistic data.

6.4 Connection to logistic regression

An easy way to understand neural networks is to begin with logistic regression. Recall that logistic regression computes a weighted sum of the input features and then applies the sigmoid function to produce a probability. For example, suppose we want to predict whether a learner belongs to a high-proficiency group (1) or a low-proficiency group (0) based on two linguistic features:

- Lexical diversity (x_1)
- Syntactic complexity (x_2)

The model assigns a weight to each feature and combines them with a bias term:

$$z = 2x_1 + 3x_2 - 2$$

Suppose a learner has a lexical diversity score of 0.5 and a syntactic complexity score of 0.6 (assuming both features are scaled between 0 and 1). The weighted sum is:

$$z = 2(0.5) + 3(0.6) - 2 = 0.8$$

The sigmoid function then converts this value into a probability:

$$P(y = 1 \mid \mathbf{x}) = \sigma(0.8) = \frac{1}{1 + e^{-0.8}} \approx 0.69$$

The model predicts a 69% probability that the learner belongs to the high-proficiency group.

A single neural unit with a sigmoid activation does exactly this. In that sense, logistic regression can be viewed as a neural network with just one computational unit and no hidden layer. Neural networks extend the logic of linear models. A single unit performs a familiar linear-plus-nonlinear computation. A larger network combines many such units into layers, allowing the model to learn increasingly abstract and useful representations.

The major practical difference is that logistic regression traditionally depends on manually engineered features, whereas neural networks often learn intermediate features automatically. For NLP, this means that instead of hand-coding feature templates such as "previous word is *not*" or "token ends in *-ing*", we can build a model that receives basic input representations and learns which combinations matter for the task.

6.5 How neural networks learn

Given some input, each layer performs a computation and passes its output to the next layer until the network produces a final prediction. This process is called a *forward pass*. At the beginning of training, the weights are typically initialized with values that do not yet encode useful patterns (much like the randomly initialized word embeddings in word2vec models). Through training,

these weights are gradually adjusted to capture patterns in the data. The network therefore needs to gradually adjust them based on examples for which the correct answer is known. The basic learning process can be understood as a repeated cycle:

make a prediction → measure the error → trace the error backward → update the weights

Consider a simple sentiment-classification example. Suppose the network receives the sentence:

The movie was surprisingly good.

Its task is to predict whether the sentence expresses positive or negative sentiment. The correct answer is *positive*. Here, P stands for *probability*, or the model's estimated likelihood of each possible class. Suppose the network initially assigns:

$$P(\text{positive}) = 0.30$$

$$P(\text{negative}) = 0.70$$

The network has made a poor prediction. Learning means using this error to determine how the weights should change so that the network is more likely to make the correct prediction next time.

Step 1: Make a prediction

The input passes forward through the network. At each layer, every unit computes a weighted sum of its inputs, applies an activation function, and passes the resulting activation to the next layer. This is repeated across multiple units and layers (because now we suppose that we are working with multiple neurons, not a single one like the one explained in the logistic regression example). At this point, the network is simply using its *current* weights to make a prediction. No learning has happened yet.

Step 2: Measure how wrong the prediction is

Next, the network compares its prediction $\hat{y}$ with the correct answer y. A *loss function* takes these two values as inputs and calculates a numerical value called the *loss*, which measures how poorly the model's prediction matches the correct answer.

$$\underbrace{(\hat{y}, y)}_{\substack{\text{Prediction and} \\ \text{correct answer}}} \xrightarrow{\text{Loss function}} \underbrace{\mathcal{L}(\hat{y}, y)}_{\text{Loss}}$$

Intuitively, a smaller loss indicates that the prediction better matches the correct answer,

while a larger loss indicates a greater mismatch. During training, the network adjusts its weights to minimize the loss. For example, if the correct answer is *positive*, predicting $P(\text{positive}) = 0.95$ should result in a much smaller loss than predicting $P(\text{positive}) = 0.30$. For classification tasks, neural networks often use a loss function such as *cross-entropy loss*, although other loss functions can also be used depending on the task and model.

For binary classification, cross-entropy loss is calculated as:

$$\mathcal{L}(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]$$

where y is the correct label (0 or 1) and $\hat{y}$ is the predicted probability of the positive class.

When the correct answer is positive ($y = 1$), the formula simplifies to:

$$\mathcal{L}(1, \hat{y}) = -\log(\hat{y})$$

Therefore, for our two predictions:

$$P(\text{positive}) = 0.95 \quad \rightarrow \quad \mathcal{L} = -\log(0.95) \approx 0.051$$

$$P(\text{positive}) = 0.30 \quad \rightarrow \quad \mathcal{L} = -\log(0.30) \approx 1.204$$

The second prediction receives a much larger loss because the model assigns a relatively low probability to the correct class.

Step 3: Determine which weights contributed to the error

Knowing that the prediction was wrong is not enough. A neural network may contain thousands, millions, or even billions of weights. The network needs to determine how each weight contributed to the final error. This is the role of *backpropagation*. The key idea can be understood intuitively by imagining that we slightly change one weight in the network. This sensitivity is expressed mathematically by a *gradient*. A gradient tells us how a small change in a weight would affect the loss. For a particular weight w, we write this relationship as:

$$\frac{\partial L}{\partial w}$$

where L represents the loss.

Here, the challenge is that weights in early layers do not directly produce the final prediction. Their outputs pass through many later computations before reaching the output layer. Backpropagation solves this problem by starting with the error at the output and working *backward* through the network. Using the chain rule from calculus, it calculates how much each earlier parameter contributed to the final loss. Conceptually, the process looks like this:

$$\text{input} \rightarrow \boxed{\text{Layer 1}} \rightarrow \boxed{\text{Layer 2}} \rightarrow \text{prediction} \rightarrow \text{loss}$$

during the forward pass, followed by:

$$\text{loss} \rightarrow \boxed{\text{Layer 2}} \rightarrow \boxed{\text{Layer 1}}$$

during backpropagation. The point is that information moves in two different directions for two different purposes. During the *forward pass*, activations move from the input toward the prediction. During *backpropagation*, information about the error moves backward so that the network can compute gradients for its parameters.

Step 4: Update the weights

Backpropagation tells us how each weight contributes to the error, but it does not itself change the weights. An *optimization algorithm* uses the gradients computed through backpropagation to update the model's parameters. One basic optimization method is *gradient descent*.

A simplified weight update can be written as:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

where η is the *learning rate*, and $\frac{\partial L}{\partial w}$ represents how the loss L changes with respect to the weight w. The learning rate controls how large each update should be. Intuitively, the network asks: which direction should I change this weight to reduce the error, and how large should that change be? The gradient provides the direction, while the learning rate determines the size of the step.[15]

Step 5: Repeat the process

Changing the weights once is not enough to train a useful neural network. The process is repeated over many training examples:

[15] It might be important to distinguish between *model parameters* and *hyperparameters*. Model parameters, such as weights and biases, are learned from the training data through optimization. In contrast, hyperparameters control how the model is structured or trained. For example, the learning rate (η) is a hyperparameter that determines how large each weight update should be. Other examples include the batch size (how many training examples are processed before updating the weights), the number of training epochs (how many times the model goes through the entire training dataset), and the choice of optimization algorithm (e.g., SGD or Adam, which determine how the weights are updated using the calculated gradients).

So, how do we choose appropriate hyperparameters? Researchers may select them based on previous studies or systematically search for suitable values through *hyperparameter tuning*. Methods such as grid search, random search, and Bayesian optimization evaluate different configurations, typically using performance on a validation dataset. The goal is to identify a configuration that generalizes well to unseen data, rather than simply minimizing the training loss.

Forward pass → Calculate loss → Backpropagation → Update parameters → Repeat

Each time, the network makes predictions using its current parameters, measures its errors, calculates how the parameters contributed to those errors, and adjusts them. Over many repetitions, useful patterns can emerge in the learned weights and hidden representations. For example, in a sentiment-classification task, the model may gradually learn representations that help distinguish expressions associated with positive and negative sentiment. Importantly, these representations do not need to be specified entirely by the researcher in advance. They can emerge as the network adjusts its parameters to perform the prediction task.

Chapter Summary

Neural networks are computational models composed of many small units that transform numerical inputs into outputs. A single neural unit takes a set of input values, multiplies each by a learned weight, adds a bias term, and computes a weighted sum $z = \mathbf{w}^\top \mathbf{x} + b$. It then applies a nonlinear activation function to produce an output $a = f(z)$. This nonlinearity is essential because it allows networks to model more complex relationships than purely linear models. A single sigmoid unit is closely related to logistic regression, but neural networks extend this idea by combining many units into layers. Unlike traditional feature-based models that rely heavily on manually engineered features, neural networks can often learn useful representations directly from relatively raw inputs.

Activity: Feature Engineering vs. Neural Networks

1. Choose an NLP task relevant to language research (e.g., predicting L2 proficiency from learner texts).
2. Feature engineering: What linguistic features would you extract to predict the outcome?
3. Neural networks: What would the model take as input, and what patterns might it learn automatically rather than relying on predefined features?
4. Compare the two approaches: What are their potential strengths and limitations in terms of (1) predictive performance, (2) interpretability, (3) linguistic insight, and (4) data size?

7 Dependency parsing and tagging

References

- de Marneffe et al. (2021). Universal dependencies. In *Computational Linguistics*. *47*(2), 255–308. [Section 7.1]
- Zeman (2025). Corpus-based language comparison: From morphology to dependencies and beyond. In *Estudos Linguísticos*. [Section 7.2]

Universal Dependencies (UD) is an annotation framework for representing the morphosyntactic structure of language in a way that can be applied consistently across many languages. The key motivation behind UD is cross-linguistic comparability. Languages can express similar meanings using different combinations of word order, morphology, and function words. For example, grammatical information that is expressed through a separate word in one language may be expressed through an inflectional marker in another. A syntactic relation that is strongly associated with word order in one language may instead be indicated primarily through case marking in another. In this sense, if every language were annotated using an entirely language-specific grammatical system, comparisons across languages would be difficult. UD attempts to solve this problem by providing a shared morphosyntactic representation while still allowing genuine differences among languages to remain visible.

7.1 Basics of UD

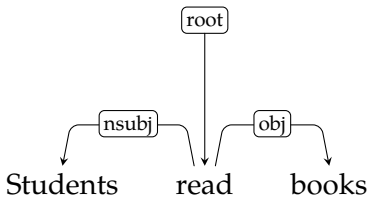
7.1.1 Dependency structures

UD is based on *dependency grammar*. In a dependency representation, syntactic structure is described primarily through relationships between words rather than through nested phrases/constituents. Each relationship connects two words: a *head* and a *dependent*. The head is the word that is structurally central to the relation, while the dependent is the word whose grammatical role is defined relative to that head.

Consider the sentence:

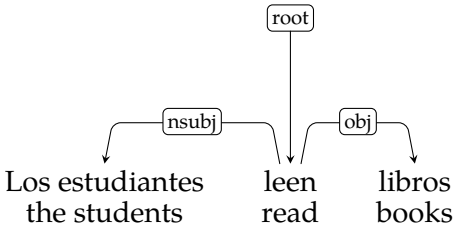
`Students read books.`

The verb **`read`** is the predicate of the clause and serves as the *root*, the topmost word in the dependency structure. The noun **`students`** depends on **`read`** as its nominal subject, while **`books`** depends on it as its object:

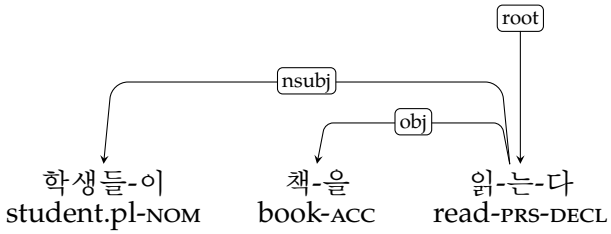


A major advantage of this representation is that comparable grammatical relations can be identified even when languages differ in word order and morphology. For example, compare the same sentence in Spanish and Korean:

Spanish



Korean



7.1.2 Words as the basic units

An important design choice in UD is that *words* receive primary status. Words carry morphological properties and enter into syntactic dependency relations with other words. This reflects what is called the *lexical integrity principle*: the internal organization of words belongs primarily to morphology, whereas syntactic structures are constructed through relations among words. For example, an inflected verb may contain information about tense, person, number, or other grammatical categories. UD can represent these properties through morphological annotation, but the internal morphemes are not normally treated in the same way as independent syntactic words. Syntax instead connects the resulting word to other words in the sentence.

This distinction appears straightforward in principle, but identifying a word is not always straightforward across languages, as we discussed earlier in Chapter 4 (Unit of analysis). Grammatical markers may develop historically from independent content words into function words, clitics, and affixes. Consequently, the boundary between a separate syntactic word and a morphological element may sometimes be difficult to determine. A computational token does not automatically correspond to a theoretically meaningful word. UD therefore has to make explicit decisions about which elements count as syntactic words before dependency relations can be assigned. These decisions are particularly important for morphologically rich languages and for cross-linguistic comparison, where different orthographic traditions may segment comparable linguistic material differently.

7.1.3 Part-of-speech categories

UPOS. Once words have been identified, UD assigns each syntactic word a *universal part-of-speech* (UPOS) category. These categories remain relatively close to familiar grammatical classes such as nouns, verbs, adjectives, and adverbs, while introducing distinctions intended to support cross-linguistic annotation. UD distinguishes 17 coarse-grained UPOS categories, summarized in Table 4.

Traditional POS	UPOS tag	Category
noun	NOUN	common noun
	PROPN	proper noun
verb	VERB	main verb
	AUX	auxiliary verb or other tense, aspect, or mood particle
adjective	ADJ	adjective
	DET	determiner (including article)
	NUM	numeral (cardinal)
adverb	ADV	adverb
pronoun	PRON	pronoun
preposition	ADP	adposition (preposition/postposition)
conjunction	CCONJ	coordinating conjunction
	SCONJ	subordinating conjunction
interjection	INTJ	interjection
–	PART	particle (special single-word markers in some languages)
–	X	other (e.g., words in foreign-language expressions)
–	SYM	non-punctuation symbol (e.g., a hash (#) or emoji)
–	PUNCT	punctuation

Table 4: Universal part-of-speech tags (UPOS). Adapted from de Marneffe et al. (2021)

XPOS. In addition to UPOS, UD allows each syntactic word to receive a *language-specific part-of-speech* (XPOS) tag. While UPOS provides a small, shared inventory for cross-linguistic comparison, XPOS can preserve finer distinctions that are useful for describing individual languages.

For English, XPOS commonly follows the *Penn Treebank POS tagset*.[16] This tagset makes distinctions that are collapsed in UPOS. For example, English nouns may be distinguished as singular or mass nouns (NN), plural nouns (NNS), singular proper nouns (NNP), and plural proper nouns (NNPS). Similarly, several English verb forms receive distinct tags, such as VB for the base form, VBD for past tense, VBG for the gerund or present participle, and VBN for the past participle.[17]

Not surprisingly, different languages require a different set of language-specific distinctions. For example, Korean-specific POS annotation distinguishes lexical stems from grammatical mor-

[16]See the Penn Treebank POS tagging guidelines (3rd revision), based on the tagset developed for the Penn Treebank project in the early 1990s: https://catalog.ldc.upenn.edu/docs/LDC99T42/tagguid1.pdf.

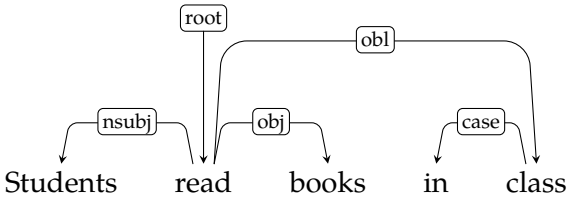
[17]For a description of the English Penn POS tags, with examples and annotation notes, you may refer to the L2 Speech POS and Dependency Annotation Project annotation manual.

phemes such as case particles and verbal endings.[18]

7.1.4 Grammatical relations

Once syntactic words and their POS categories have been identified, UD represents the syntactic relationships between words using *grammatical relations*. In a dependency relation, each dependent is connected to its syntactic head by a grammatical relation. UD provides a universal inventory of 37 basic grammatical relations that is intended to be applicable across languages (Table 5).

A particularly important distinction in UD is that between *core* arguments and *oblique* dependents. Nominal subjects and objects are treated as core arguments and are generally represented with relations such as `nsubj`, `obj`, and `iobj`. Nominals that modify a predicate but are not analyzed as core arguments are normally represented with `obl`. Consider, for example:



Here, `students` and `books` are core arguments of `read`, whereas `class` is an oblique dependent expressing the location of the event. Also note that UD analyzes the preposition `in` as a dependent of the nominal `class`, using the relation `case`. This reflects a general UD analysis in which adpositions and morphological case markers are treated as alternative strategies for marking the grammatical role of nominals.

7.2 Some implications of UD

UD is not intended to be a complete theory of grammar in the same sense as many theoretical linguistic frameworks. Its design priorities are partly practical. The annotation needs to be linguistically motivated but also sufficiently transparent and consistent that it can be used by researchers working in linguistics, psychology, computer science, language education, and related areas.

At the center of the framework is a balancing problem. If annotations are made too language-specific, comparison becomes difficult because equivalent constructions receive unrelated representations. If annotations are made too universal, however, genuine differences among languages may disappear. UD therefore attempts to find a middle ground. Comparable morphosyntactic phenomena should be represented similarly where possible, but language-specific distinctions should not be erased merely to create superficial uniformity. This principle is also important when interpreting UD data. A shared annotation label does not imply that two languages realize a construction in exactly the same way, but may indicate that it has been analyzed as comparable at the level represented by the annotation scheme.

[18] For a description of the Korean Sejong tag set, with examples and annotation notes, you may refer to the UD-KSL repository.

Relation	Description
acl	Clause modifying a nominal
advcl	Adverbial clause modifying a predicate or another modifier
advmod	Adverbial modifier
amod	Adjectival modifier of a nominal
appos	Nominal in an appositional relation with another nominal
aux	Auxiliary expressing grammatical information such as tense, aspect, mood, or voice
case	Case-marking element associated with a nominal, including adpositions
cc	Coordinating conjunction associated with a conjunct
ccomp	Clausal complement without obligatory control of its subject
clf	Classifier associated with a nominal or numeral
compound	Relation between elements forming a compound
conj	Relation connecting coordinated elements
cop	Copular element connecting a nonverbal predicate with its subject
csubj	Clause functioning as the syntactic subject of a predicate
dep	Unspecified dependency used when a more precise relation cannot be assigned
det	Determiner modifying a nominal
discourse	Discourse element such as an interjection or discourse marker
dislocated	Peripheral nominal related to a clause, such as a topic or afterthought
expl	Expletive element without an ordinary semantic argument role
fixed	Relation between elements of a fixed grammaticalized expression
flat	Relation between elements of a headless or semi-fixed multiword expression
goeswith	Relation connecting parts of a word that are separated orthographically
iobj	Indirect object or other secondary nominal core argument
list	Relation connecting elements presented as a list
mark	Marker introducing or signaling a subordinate clause
nmod	Nominal modifier of another nominal
nummod	Numeric modifier of a nominal
nsubj	Nominal functioning as the syntactic subject of a predicate
obj	Primary non-subject nominal core argument of a predicate
obl	Nominal functioning as a non-core modifier of a predicate
orphan	Relation used for dependents whose expected head is omitted through ellipsis
parataxis	Relation between clauses or phrases placed alongside one another without ordinary coordination or subordination
punct	Punctuation attached to the head of a clause or phrase
reparandum	Element involved in the repair of a spoken-language disfluency
root	Relation identifying the root of the dependency tree
vocative	Nominal referring directly to an addressee
xcomp	Clausal complement whose subject is obligatorily controlled by another argument

Table 5: The 37 basic grammatical relations in Universal Dependencies.
Adapted from de Marneffe et al. (2021).

7.2.1 UD as a corpus resource

The practical value of UD lies not only in its annotation framework but also in the growing collection of annotated treebanks. At a minimum, UD treebanks provide linguistically annotated (i.e., gold-annotated) and/or manually verified (i.e., silver-annotated) POS information and dependency relations that connect words into syntactic trees. The collection includes data from a wide range of genres and sources, such as news, Wikipedia, fiction, religious texts, social media, and spoken language. For some languages, multiple treebanks are available, often representing different genres, domains, or data sources.[19]

These resources give researchers direct access to syntactically annotated corpora and therefore support research questions extending well beyond parser development. For example, a researcher interested in relative clauses can identify relevant dependency structures (rather than relying solely on lexical cues such as *who*, *which*, or *that*). Similarly, a researcher examining verb–argument patterns can retrieve verbs together with dependents annotated as `nsubj`, `obj`, or `obl`. In this sense, UD provides a searchable empirical resource for corpus-based linguistic research.

7.2.2 UD and learner corpora

UD has also been increasingly applied to second language (L2) research. A number of UD treebanks have been developed from learner language, allowing researchers to investigate morphological and syntactic patterns in non-native production within a standardized annotation framework. These resources now cover several target languages and both written and spoken modalities, as summarized in Table 6.

Target language (modality)	Reference
Chinese (written)	Lee et al. (2017)
English (written)	Berzak et al. (2016)
English (spoken)	Kyle et al. (2022)
Italian (written)	Di Nuovo et al. (2019)
Korean (written)	Sung and Shin (2024)
Russian (written)	Rozovskaya (2024)
Spanish (written)	Pulido et al. (2025)
Swedish (written)	Volodina et al. (2025)

Table 6: Examples of UD-annotated L2 learner corpora

Suppose, for example, that we want to compare how English learners at different proficiency levels use verbal complements. A surface-based analysis might search for particular complementizers or recurring lexical sequences, such as think that, *want to*, or *wonder whether*. This approach can miss cases in which the same broad syntactic relationship is realized with different words or constructions. A dependency-based analysis can instead retrieve complements according to their grammatical relation to the governing verb. For instance, UD distinguishes clausal complements

[19] For example, UD English GUM.

such as `ccomp` in *I think* [*that she is right*] and open clausal complements such as `xcomp` in *I want* [*to leave*]. By doing this, researchers might be able to compare how frequently or diversely learners use these complement relations without restricting the analysis to a predetermined set of complementizers, verbs, or surface word sequences.

At the same time, learner data raises annotation questions. Learners may produce non-target-like word forms or syntactic patterns that do not match structures typically described in reference grammars. When annotating learner language, researchers must therefore decide to what extent existing annotation conventions can be generalized to learner-produced structures and how to represent constructions that deviate from target-language norms.

7.2.3 Corpus-based language comparison

One of the major advantages of UD is that its shared annotation scheme facilitates large-scale cross-linguistic comparisons. Traditional linguistic typology often classifies languages into categories such as SVO or SOV based on their dominant word order. Corpus data, however, allows us to quantify word-order preferences. Suppose we want to investigate the relative order of subjects, verbs, and objects. Because UD identifies subjects and objects through dependency relations, we can calculate how frequently each argument occurs before or after its predicate. We can also examine the distribution of different word orders in clauses where both subjects and objects are explicitly expressed. A language traditionally classified as SOV may show a strong preference for this order without using it exclusively, whereas another language may exhibit substantially greater variability. We can therefore move beyond categorical questions (e.g., Is this language SVO or SOV) toward probabilistic questions (e.g., How strongly does this language prefer particular word orders in actual language use?).

7.2.4 Comparability does not guarantee representativeness

The existence of a common annotation scheme does not automatically make every comparison valid. An important limitation of UD is that the collection is uneven across languages, language families, corpus sizes, and genres. Some well-resourced languages have multiple large treebanks, whereas other languages may be represented by only small datasets. The types of texts may also differ. One language might be represented mainly through newspaper texts, while another might be represented through religious texts or spoken fieldwork data. Suppose we observe that one language contains a much higher frequency of a particular syntactic construction than another. Several explanations are possible:

1. the languages genuinely differ in their grammatical preferences;
2. the treebanks represent different genres or registers;
3. the corpora differ substantially in size or composition; or
4. annotation practices differ in ways that affect the counts.

A shared annotation framework solves only part of the comparability problem. Good corpus research still requires careful corpus design, metadata inspection, sampling decisions, and consideration of register and genre, which is important for typological research. UD's language coverage is broad, but the collection was not constructed as a balanced typological sample.

Chapter Summary

Universal Dependencies (UD) provides a framework for morphosyntactic annotation across languages. It is an annotation scheme and a collection of syntactically annotated corpora known as treebanks. Its goal is to represent comparable grammatical structures in comparable ways while preserving genuine differences in how languages encode those structures.

Activity: Practicing UD annotation

The materials for this activity will be provided in the course repository. The sentences have already been processed by a UD parser and are therefore *silver-annotated*. Your task is to inspect these annotations and correct them where necessary.

Procedure

1. Open the silver-annotated CoNLL-U file provided in the repository.
2. Read each sentence first as ordinary text and consider its syntactic structure before examining the parser's analysis.
3. Inspect the annotation token by token. In particular, check:
   - whether the tokenization is appropriate;
   - whether the lemma is correct;
   - whether the UPOS and XPOS categories are appropriate;
   - whether the morphological features are correct;
   - whether each token is attached to the correct head; and
   - whether the dependency relation accurately describes the relation between the token and its head.
4. When you identify an annotation error, record the annotation you consider more "appropriate", based on your linguistic judgment, in a separate column.

As you work, keep track of the types of annotation errors you encounter. Consider whether they primarily involve POS tagging, morphological analysis, head selection, dependency relations, or particular syntactic constructions. Reflect on any recurring patterns, as these observations will help you evaluate both the usefulness and the limitations of automatically annotated corpora.

8 Measuring constructs in SLA

References

- Bulté et al. (2025). Complexity and difficulty in second language acquisition: A theoretical and methodological overview. In *Language Learning*. *75*(2), 533–574.

In SLA research, many phenomena of interest are conceptualized as "theoretical" constructs that cannot be directly observed and therefore require explicit definitions and operationalization for empirical investigation. These constructs provide a basis for describing, measuring, and explaining different aspects of second language development and performance. Among them, linguistic *complexity* has received considerable attention in SLA and applied linguistics for both theoretical and applied reasons (e.g., Bulté and Housen, 2012; Norris and Ortega, 2009; Pallotti, 2015). From a theoretical perspective, measures of complexity can contribute to the description and comparison of linguistic structures across languages and varieties and can provide evidence relevant to theories of language structure, language evolution, and human language processing. From a more applied perspective, complexity has been investigated in areas such as language development, language assessment, pedagogy, and clinical contexts, where researchers often seek accurate and reliable measures of language performance, proficiency, or development. Despite this broad relevance, however, the concept of *complexity* has been used in many different ways. Different studies have used the same term to refer to structural elaboration, cognitive processing demands, developmental timing, or characteristics associated with higher levels of language proficiency. At the same time, a large number of measures have been proposed under the general label of complexity.

The goal of this chapter is to clarify what researchers mean by *complexity* and *difficulty* and how these constructs can be operationalized. In particular, Bulté, Housen, and Pallotti (2025) propose that complexity and difficulty should be treated as conceptually distinct constructs. The discussion focuses on the formal properties of linguistic structures rather than their communicative functions. The term *structure* or *item* is used for relatively abstract linguistic objects, such as syntactic structures, lexical items, or form–function pairings, while *form* or *unit* may refer to concrete instantiations of such structures. The term *text* is used broadly to refer to both spoken and written language productions containing these forms and units. Complexity and difficulty can be examined either (1) at the level of individual linguistic structures or (2) at the level of entire texts. The chapter also follows the common practice in SLA of discussing different linguistic levels, particularly the lexicon, morphology, and syntax. These distinctions are analytically useful, although the boundaries between linguistic levels are not always clear-cut. As will become important later in this chapter (and as we briefly discussed in Chapter 4 in relation to the definition of a word) even seemingly basic linguistic units require explicit theoretical and operational definitions.

8.1 From complexity as an umbrella term to distinct constructs

8.1.1 Complexity in the CAF framework

The term *complexity* was used occasionally in early SLA research to characterize aspects of second language development (e.g., Larsen-Freeman, 1978). It later became particularly influential as one component of the complexity–accuracy–fluency (CAF) framework (Norris and Ortega, 2009). One motivation for the CAF framework was the recognition that second language development cannot be adequately described simply as increasing accuracy, understood as increasingly target-like language use. The concept of *interlanguage*, for example, emphasizes that learner language constitutes a developing linguistic system. Such a system may expand by providing learners with a wider range of linguistic options even when those options do not necessarily result in greater grammatical correctness. At the same time, learners may become increasingly fluent as access to linguistic knowledge becomes more automatized. In this sense, complexity was adopted as a useful label for dimensions of language development that could not be captured by accuracy and fluency alone. The resulting three-way distinction can be summarized broadly as follows:

- **Accuracy**: the extent to which language use conforms to target-language norms
- **Fluency**: the efficiency and automatization with which language is processed or produced
- **Complexity**: the expansion and elaboration of the linguistic resources available to a learner. (One challenge in defining complexity is that this broad conceptualization does not specify which aspects of language are being elaborated or how such elaboration should be measured.)

8.1.2 Different meanings of complexity

A number of theoretical discussions have questioned the use of *complexity* as an umbrella term for essentially all dimensions of language performance other than accuracy and fluency. In particular, researchers have distinguished at least two fundamentally different ways in which something can be described as complex. The first concerns properties of the linguistic object itself. From this perspective, complexity refers to the structural characteristics of an object, phenomenon, or system. Rescher (1998), for example, characterizes complexity in terms of the number and variety of constituent components and the elaborateness of the relationships among them. Applied to language, a linguistic structure or text can be described as more or less complex on the basis of its structural organization. A second use concerns the interaction between a linguistic object and a human learner or language user. A linguistic structure may require greater cognitive resources to learn, comprehend, or produce. In this case, what is being described is not primarily a property of the linguistic structure itself but the *difficulty* that the structure presents to an individual.

In SLA research, the term complexity has additionally become associated with developmental order and proficiency. A structure that is acquired later may be described as more complex, on the assumption that later acquisition indicates greater learning difficulty. Similarly, structures that occur more frequently in the language of advanced learners may be characterized as complex

on the assumption that greater complexity is associated with higher proficiency. As a result, the term *complex* can potentially mean several different things: (1) structurally elaborate, (2) cognitively demanding, (3) acquired relatively late, or (4) characteristic of more proficient language use.

These properties may be empirically related, but they do not necessarily represent the same construct (i.e., the underlying theoretical concept being described or measured). Treating them as interchangeable creates the risk that measures are selected simply because previous research has labeled them "complexity measures," without sufficient consideration of the theoretical relationship between the empirical measure and the construct it is intended to represent (Bulté and Housen, 2012).

8.2 Defining complexity and difficulty

To address this problem, Bulté, Housen, and Pallotti propose distinguishing between *complexity* and *difficulty*. They also briefly discuss *development* and *proficiency*, and how these concepts are related, but this lecture will focus primarily on complexity and difficulty.

8.2.1 Complexity

Definition. Bulté, Housen, and Pallotti reserve the term *complexity* for the structural characteristics of linguistic items, structures, and texts. More specifically, complexity concerns the quantity and variety of linguistic components and the relationships among those components. This definition overlaps with Rescher's (1998) notions of compositional and structural complexity and is broadly consistent with Biber et al. (2020), who characterize complexity in terms of the structural elaboration of linguistic units.

What is complexity a property of? Complexity is treated as a property of the linguistic object itself. The question is what a word, morphological form, syntactic structure, or text is structurally like. For example, a researcher may ask:

- How many components does the linguistic form contain?
- How varied are those components?
- How many relationships hold among the components?
- How hierarchically elaborate are those relationships?

A syntactic structure with more constituents, more levels of embedding, or a more elaborate dependency structure may therefore be described as structurally more complex. At the lexical or morphological level, complexity may similarly concern the number and variety of elements, patterns, or operations involved.

What contributes to complexity? Given this definition, different dimensions of structural complexity can be distinguished. These include, for example (See Table 7):

- constitutional complexity: the number of components making up a linguistic unit
- taxonomical complexity: the variety of different components or structural types
- hierarchical complexity: the depth or organization of relationships among components
- organizational complexity: the distribution and arrangement of linguistic structures across a larger text

Complexity dimension	Constitutional	Taxonomical	Hierarchical	Organizational
Lexicon	*Mean word length*	MATTR of lemmas *Entropy-based measures* *Yule's K*		
Morphology	*Mean number of morphological operations*	MCI		
Syntax	Mean words per phrase Mean phrases per clause Mean clauses per T-unit or AS-unit *Number of nodes in tree* *Number of dependents per syntactic unit* *Mean words per (finite) clause, T-unit, AS-unit*		Normalized rate of occurrence of dependent clauses and phrases *Maximum depth of syntactic tree* *Dependency distance* *Hierarchical distance*	MATTR of dependency relations or syntactic structures *Mean sentence similarity score*
Lexico-grammatical		MATTR of lexico-grammatical units		

Table 7: Core and noncore measures of complexity. Adapted from Bulté et al. (2025, p. 559).
Note. Noncore measures are shown in italics, which are not necessarily less appropriate but tend to address highly theory-specific research questions, have relatively limited empirical support, or be difficult to implement in practice.

What does complexity NOT mean? Several concepts often discussed under the general label of "complexity" fall outside this definition. First, complexity does not include *sophistication* when sophistication is defined primarily in terms of frequency or rarity (e.g., Ortega, 2003). Frequency describes how often an item occurs in language use; it is not part of the item's structural makeup. A rare word is not necessarily structurally more complex than a frequent word. Second, complexity does not refer to *communicative appropriateness*. A structurally elaborate form is not necessarily better suited to a particular genre, register, or communicative purpose. A relatively simple structure may in fact be the most appropriate choice in a given context. Third, complexity does not itself refer to the *cognitive effort* required to process or learn a linguistic structure. Structural elaboration may contribute to processing or learning difficulty, but complexity and difficulty remain distinct constructs.

What kind of evidence is needed? Because complexity concerns an observable or describable property of linguistic structure, establishing its degree does not necessarily require data from learners or language users. Instead, its measurement depends primarily on the descriptive adequacy, internal coherence, and transparency of the linguistic analysis being adopted.

Researchers must specify clearly what counts as a linguistic unit, what counts as a component, and what kinds of relationships among components are being measured. For example, a study using clause length as a complexity measure must explain what counts as a clause; a lexical complexity study must specify what counts as a word or lexical unit.

How does complexity relate to difficulty? Complexity concerns primarily the "what" of acquisition: the structural properties of the linguistic material that learners encounter and eventually acquire. Structural complexity may contribute to difficulty. For example, a longer or more deeply embedded syntactic structure may impose greater memory or integration demands. However, the structural property itself and the cognitive cost associated with processing it are not the same thing.

8.2.2 Difficulty

Definition. Bulté, Housen, and Pallotti use *difficulty* to refer to the cognitive demands associated with learning or processing linguistic structures. A linguistic structure can be described as more difficult when its learning, comprehension, or production requires greater cognitive resources (e.g., attention, memory, processing time, or mental effort) from a language learner or user in a particular context. The concept is related to the broader notion of *computational complexity*, the amount of effort required to solve a problem, whether by a human being or by a computational system.

What is difficulty a property of? Unlike complexity, difficulty is not treated as a property of the linguistic structure alone. It is a relational property arising from the interaction among the linguistic structure, the context in which it is encountered, and the learner or language user processing it. The relevant question is what it requires from a particular learner or user under particular conditions. For example, a researcher may ask:

- How much time does a structure take to process?
- How much cognitive effort does its comprehension or production require?
- How accurately can learners comprehend or produce it?
- How readily or late is the structure learned?

What contributes to difficulty? Three broad sources of difficulty have been identified in the previous literature (e.g., DeKeyser, 2005; Housen and Simoens, 2016):

- Structure-related factors: Structure-related factors can include properties such as length, structural elaboration, salience, and transparency. For instance, longer syntactic dependencies may require more information to be maintained and integrated during processing, while opaque form-meaning mappings may be harder to learn than transparent one-to-one mappings.
- Context-related factors: Context-related factors include characteristics of learners' experience with the structure. One particularly important variable is frequency: structures encountered

more frequently in the input may be easier to learn or process. Other relevant variables include dispersion across contexts and association strength between linguistic elements.

- Learner-related factors: Learner-related factors include differences in prior linguistic knowledge, proficiency, experience with the target language, cognitive resources, and other individual characteristics. Thus, the same structure may be relatively easy for one learner but difficult for another.

Complexity dimension	Structure-related	Context-related	Evidence for difficulty
Lexicon	Average word length *Derivational compositionality* *Abstractness, imageability*	Average frequency in reference corpus	*Age of L1 acquisition*
Morphology	*Transparency/regularity* *Saliency*	*Communicative value*	Stage of acquisition *Subjective ratings*
Syntax	Structural complexity (variously operationalized: words, nodes, dependencies, etc.) Canonicity (e.g., in Extended Processability Theory)		Stage of acquisition *Subjective ratings* *Processing cost for automated parser*
Lexico-grammatical		*Average frequency in reference corpus* *Association strength*	

Table 8: Core and noncore measures of difficulty. Adapted from Bulté et al. (2025). p. 560.

What does difficulty NOT mean? First (and again), difficulty should not be equated with structural complexity. A structurally complex form may be relatively easy to process if it is highly frequent, salient, transparent, or familiar. Conversely, a structurally simple form may be difficult if it is rare, perceptually inconspicuous, or associated with an opaque form–meaning mapping. Second, difficulty should also not be equated with development. A difficult structure may emerge or be mastered relatively late, and developmental order can therefore provide evidence about difficulty. However, late acquisition is an observed developmental outcome, whereas difficulty is one possible explanation for that outcome. Finally, difficulty should not be equated with proficiency. Proficiency is a much broader construct referring to a learner or user's ability to use linguistic and other communicative resources effectively across communicative contexts. It may involve difficulty-related abilities but also encompasses constructs such as accuracy, fluency, appropriateness, adequacy, and communicative effectiveness.

What kind of evidence is needed? Because difficulty concerns the experience of a learner or language user, it cannot generally be established by inspecting a linguistic structure alone. Claims about difficulty require a theoretical account of why a particular property should increase processing or learning demands and, ideally, empirical evidence from human learners or users showing that such costs actually occur. Difficulty can be investigated through several types of evi-

dence:

- developmental or acquisitional timing
- comprehension or production accuracy
- reaction times and other online processing measures
- eye movements, pupil dilation, ERP, or other psychophysiological measures
- subjective ratings by learners or experts

If a sufficiently strong empirical relationship is established between a particular linguistic property and observable processing or learning costs, that property may subsequently be used as an *indicator* or proxy for difficulty. For example, if longer dependency distances are consistently associated with greater processing time, dependency distance may be used as an indicator of processing difficulty. The empirical relationship, however, must first be established rather than simply assumed.

How does difficulty relate to complexity? Difficulty concerns primarily the "how" and "why" of acquisition: why some linguistic structures require more effort to process or learn and why learners may follow particular developmental trajectories. Complexity can be one source of difficulty, but it is only one among several possible contributors. Contextual and learner-related factors can increase or decrease the difficulty associated with the same structural configuration. Thus, a linguistic structure can be structurally complex without necessarily being difficult, and it can be difficult for reasons other than structural complexity.

8.3 Interactions among complexity, difficulty, development, and proficiency

Complexity and difficulty are related to L2 development and proficiency, but these relationships are not simple or linear.

Complexity and development. As learners develop, they often become able to produce and comprehend more complex linguistic structures. However, this does not mean that complexity increases continuously over time. Different levels and types of complexity may be more appropriate for different genres and communicative situations, and development may sometimes involve a shift toward less complex language. Longitudinal research also shows that complexity often develops in nonlinear ways and that developmental trajectories can vary substantially across individual learners.

Difficulty and development. Structures that are more difficult to learn or process may tend to emerge later in development. At the same time, difficulty is multidimensional: a structure may be difficult in one respect but relatively easy in another. For this reason, more empirical evidence is needed to determine the degree and sources of difficulty associated with particular structures across different languages.

Complexity and proficiency. More proficient learners often have access to a wider range of complex linguistic resources, but higher proficiency should not be equated simply with greater complexity. At advanced levels, proficient language use may involve choosing relatively simple structures when they are more appropriate for a particular communicative context. Thus, what matters is not only how much complexity a learner uses, but also what type of complexity is used and whether it is appropriate for the communicative situation.

Chapter Summary

The measurement of theoretical constructs in SLA research requires conceptual definitions and careful operationalization, as these decisions determine what is measured and how the results can be interpreted. Using linguistic complexity and difficulty as examples, this chapter illustrates how closely related but distinct constructs can capture different aspects of learner language and its development. Linguistic complexity concerns the structural properties of language, whereas difficulty refers to the cognitive effort involved in learning, processing, or using linguistic structures. The chapter also suggests that translating such abstract constructs into measurable indices requires explicit decisions about linguistic units and levels of analysis, including lexical, morphological, syntactic, and lexicogrammatical features.

9 Linguistic annotations

References

- Leech (2004). Adding linguistic annotation. In *Developing linguistic corpora: A guide to good practice*.

Corpus linguists rarely work with raw texts. Alongside a text (or a speech transcript), a corpus may contain information about speakers and texts, structural boundaries, and analyses of the language itself. In earlier chapters, we discussed how the definition of a linguistic unit affects the frequencies and distributions we observe. Annotation extends this discussion by considering what linguistic information and interpretations should be explicitly attached to those units and how they should be represented.

9.1 What is corpus annotation?

Corpus annotation is a practice of adding interpretative linguistic information to a corpus. The addition is interpretative because it moves beyond recording what a person wrote or said: it assigns a linguistically meaningful analysis to part of that data. For instance, the sentence *Dogs bark loudly* can be represented with lemmatized and POS information as follows:

Token	Dogs	bark	loudly
LEMMA	dog	bark	loudly
POS	NOUN	VERB	ADV

As you may already know from automated annotations (of lemmatization, POS tagging, dependency parsing, etc), linguistic annotation is not a single task. It can target almost any linguistic level, and several levels can coexist in the same corpus. Table 9 summarizes representative categories. These types differ also in their units of annotation. A POS tag usually attaches to an individual token. A lemma maps a token to a lexical form. A syntactic relation connects two elements; a constituent spans several tokens. An intonational contour may extend over an entire phrase, and an anaphoric relation may connect expressions separated by several sentences. In learner-language research, error annotation may target a token, a phrase, a missing item, or a longer sequence.

9.2 Benefits of annotation

9.2.1 Efficient retrieval

An important benefit of annotation is that information encoded can be extracted repeatedly. Suppose a lexicographer wants to study the word *separate*. A plain-text search retrieves both *They separate the groups* and *They live in separate houses*. With POS annotation, verb uses and adjective uses can be examined separately. The same principle applies to complex patterns. If syntactic dependencies have been annotated, a researcher can look for subjects of particular verbs or for a

Table 9: Representative types of linguistic annotation.

Type of annotation	**Description**	**Examples**
POS tagging	Adding information about the grammatical category (part of speech) of each word in a text.	Noun, verb, adjective, adverb.
Phonetic annotation	Adding information about how words or sounds are pronounced in a spoken corpus.	IPA transcription, individual speech sounds.
Prosodic annotation	Adding information about the prosodic features of spoken language.	Stress, intonation, pauses, rhythm.
Syntactic annotation	Adding information about the syntactic structure of sentences, including how words are organized into phrases and clauses.	Constituency parsing, dependency parsing.
Semantic annotation	Adding information about the meanings or semantic categories of words and expressions.	*Cricket* as a sport vs. *cricket* as an insect.
Pragmatic annotation	Adding information about the communicative functions of utterances, including speech acts and dialogue acts.	Acknowledgment, request for feedback, acceptance, pragmatic markers signaling a new phase of discussion.
Discourse annotation	Adding information about relationships between linguistic elements across sentences or larger stretches of text.	Anaphoric links between pronouns and their antecedents.
Stylistic annotation	Adding information about stylistic features, including how speech and thought are represented in a text.	Direct speech, indirect speech, free indirect thought.
Lexical annotation	Adding information about the lexical identity of words, such as the lemma associated with each word form.	*Running*, *runs*, *ran* $\rightarrow$ *run*.
Error annotation	Adding information about errors in language production, particularly in learner corpora, including their locations and types.	Grammatical errors, lexical errors, spelling errors, word-order errors.

particular verb–object relationship. If discourse functions are annotated, the researcher can compare the distribution of those functions across registers or proficiency levels.

Beyond facilitating the search and retrieval of linguistic patterns, annotations provide a foundation for downstream analyses. For example, POS tags can be used to generate frequency dictionaries organized by grammatical class, while syntactic annotations can be used to calculate measures of grammatical complexity.

Example: From linguistic annotation to automatic analysis

Original sentence

The student who arrived late submitted the assignment.

POS and dependency annotations

Token	POS	Head	Dependency
The	DET	student	det
student	NOUN	submitted	nsubj
who	PRON	arrived	nsubj
arrived	VERB	student	acl:relcl
late	ADV	arrived	advmod
submitted	VERB	ROOT	root
the	DET	assignment	det
assignment	NOUN	submitted	obj
.	PUNCT	submitted	punct

Automatic analysis using these annotations

- **POS-based frequency counts:** 2 nouns (*student, assignment*), 2 verbs (*arrived, submitted*), and 1 adverb (*late*).
- **Grammatical complexity:** 1 relative clause, identified through the dependency label `acl:relcl` (*who arrived late*).

9.2.2 Reusability of annotations

Linguistic annotation is a resource-intensive process. Because of the complexity and ambiguity of language, automatic annotation often requires additional human effort to achieve a high level of accuracy. For example, Leech (2004) discusses how the automatically assigned POS tags in the LOB corpus and the BNC Sampler were manually post-edited, whereas the full British National Corpus (BNC) was released without exhaustive manual correction because of the substantial cost involved. This investment becomes more valuable when annotated corpora can be shared and reused. Researchers can build on existing annotated resources to investigate different research questions, instead of independently annotating the same data from scratch. When such resources are made openly available under appropriate licenses (e.g., Creative Commons Attribution [CC

BY 4.0]), they can reduce duplicated effort, facilitate comparisons across studies, and support the reproducibility of linguistic research.

9.3 Principles of good annotation practice

9.3.1 Preserve and separate the original data

It should be possible to recover the original corpus without losing information when annotation is added. Annotations can be stored inline, alongside text, or stand-off, in a separate file that refers to positions or identifiers in the source. Both approaches can work, provided the original text or recording, segmentation decisions, and annotation layers remain traceable. Keeping a record of normalization is particularly important for historical texts, spontaneous speech, and learner language: an annotator's corrected version must not silently replace what was actually produced.

9.3.2 Document the annotation explicitly

Documentation should provide a reproducible account of how corpus annotations were produced. It should identify the annotation scheme and coding conventions, the tools or models used, the annotators involved, the dates and stages of annotation, annotator training procedures, revisions, and quality checks. For automatically annotated corpora, relevant details also include model names and versions, training domains where known, and any manual review or correction. For manually annotated corpora, documentation should specify whether annotators worked independently, whether they had access to existing tags, and how disagreements were resolved. A reusable corpus thus requires more than a list of labels: it requires a transparent record of the decisions and procedures behind them.

To support consistency and reusability, researchers typically develop an *annotation manual* before annotation begins and update it as new linguistic phenomena or ambiguous cases arise. The manual describes the annotation scheme, defines each category, and provides guidance for difficult or exceptional cases. Such guidance is particularly important for learner corpora, where pauses, repetitions, false starts, omitted arguments, and nonstandard word forms can complicate segmentation and grammatical interpretation. Although some annotation decisions necessarily rest on convention, explicit and consistently applied rules help annotators make comparable decisions and allow other researchers to interpret and reuse the resulting data. Examples include:

- ASC Treebank Annotation Manual
- LxGrTagger Annotation Guidelines

Shared annotation guidelines also support collaborative corpus development by giving contributors a common framework for producing compatible data. Universal Dependencies, introduced earlier in relation to dependency annotation (Chapter 7), illustrates how a shared framework

can support annotation across languages and research teams. Similarly, the CHILDES project[20] illustrates the value of common transcription and coding conventions for building a shared research resource. These conventions allow data contributed by different projects to be analyzed using compatible software and more readily compared or combined.

9.3.3 Prefer interpretable and consensual categories

There is no theory-independent "God's truth" annotation for all linguistic phenomena. Linguists often disagree about category boundaries, especially in less prototypical cases. Nevertheless, they may agree sufficiently about core instances of nouns, verbs, clauses, or grammatical relations to build a useful scheme. The aim is to identify distinctions that other researchers can understand and that annotators can apply in a reasonably consistent way. Consensus is not the same as pretending that all expressions are clear-cut. A rigorous scheme should explain how difficult cases are handled. For example, should a multifunctional word be tagged by its dictionary entry or by the context-specific grammatical function it performs? How should an incomplete learner utterance be treated? Operational definitions make those choices visible and reviewable.

9.3.4 Respect established practice without treating it as immutable

New projects should examine established tagsets, manuals, and data formats before inventing another convention. Departures can be fully justified (e.g., an L2 study may need distinctions that a general-purpose L1 treebank does not encode) but should be explained so that users can understand the relationship between the new scheme and earlier ones.

9.4 Evaluating annotation quality

A good annotation project must ask at least three questions: (1)Does the scheme represent an informative linguistic distinction? and (2) Are the annotations assigned correctly and (3) consistently under that scheme?

9.4.1 Validity: are the categories linguistically meaningful?

An annotation scheme might be applied with perfect consistency and still be unhelpful, which is related to validity problem. For example, one could divide a dictionary alphabetically into arbitrary groups and achieve highly reproducible labeling. Such categories would not, however, tell us much about grammar or lexical meaning. Linguistically useful POS categories should reflect some affinity in syntactic distribution, morphological behavior, or semantic interpretation; a discourse annotation should reflect a defensible theory of communicative function. Reliable annotation is necessary for many applications, but reliability does not establish that a construct has been operationalized well.

[20] https://talkbank.org/childes/

9.4.2 Accuracy: how often are reference labels recovered?

Accuracy is the proportion of evaluated units for which the assigned labels match the "gold" labels:

$$A = \frac{C}{N} \tag{1}$$

where C is the number of correctly labeled units and N is the total number of evaluated units.

Precision and recall reveal different types of error. Suppose `NOUN` is the target category. A true positive (TP) is a noun correctly predicted as a noun; a false positive (FP) is a non-noun incorrectly predicted as a noun; and a false negative (FN) is a noun that the tagger fails to identify as a noun.

Precision (P) is calculated as:

$$P = \frac{TP}{TP + FP} \tag{2}$$

Recall (R) is calculated as:

$$R = \frac{TP}{TP + FN} \tag{3}$$

The F1 score is the harmonic mean of precision and recall:

$$F_1 = \frac{2PR}{P + R} \tag{4}$$

Precision asks, "Of everything the model labeled as a noun, how much really is a noun?" Recall asks, "Of everything that really is a noun, how much did the model find?" The F1 score is their harmonic mean. Accuracy considers correct decisions across all evaluated categories, whereas category-specific precision and recall reveal how a particular label behaves.

Worked example. Imagine a corpus containing exactly 100 noun tokens. One conservative model labels only 50 tokens as nouns, of which 48 are correct. Its noun precision is $48/50 = .96$, but noun recall is only $48/100 = .48$: it rarely assigns an incorrect noun label, but it misses more than half the actual nouns. Another model labels 150 tokens as nouns and gets 95 of the 100 nouns. Its precision is $95/150 \approx .63$ and recall is $95/100 = .95$: it finds most nouns but incorrectly includes many non-nouns. Neither "high precision" nor "high recall" alone establishes that a model meets a particular research need.

For multi-category annotation, scores can be reported by tag, or aggregated in ways such as micro- or macro-averaging. The choice matters when categories differ substantially in frequency.

An overall score dominated by common nouns or verbs may conceal poor performance on rare relations, constructions, or learner-language phenomena.

9.4.3 Consistency: do annotators apply the scheme in the same way?

Inter-annotator agreement concerns independent human decisions, not model performance against a reference. Simple agreement is the proportion of annotation decisions on which the coders assign matching labels. It is interpretable but does not account for agreement expected from the frequencies of the categories. Cohen's κ adjusts for this expected agreement:

$$\kappa = \frac{P_o - P_e}{1 - P_e}, \tag{5}$$

where P_o is observed agreement and P_e is expected agreement under the coefficient's chance model. Kappa is sensitive to category prevalence and coder behavior; generic qualitative cutoffs are less informative than reporting the coefficient with the task, annotation unit, number of coders, label distribution, and uncertainty where feasible. A confusion matrix can show *which* categories are being confused even when the overall agreement appears strong.

Krippendorff's α may provide a more general chance-corrected framework that can accommodate multiple annotators, missing judgments, and different definitions of disagreement appropriate to nominal, ordinal, or other measurement levels (Hayes and Krippendorff, 2007).

Adjudication and post-editing. When annotators disagree, a subsequent discussion or additional review may produce a final label. That final decision is useful for a reference corpus, but agreement should ordinarily be calculated on the independent decisions before adjudication; otherwise the reported number hides the ambiguity that made adjudication necessary. Similarly, post-editing means that human analysts inspect and correct automatic labels. Researchers should distinguish raw model output from the revised annotations in any evaluation or release.

Chapter Summary

Linguistic annotation adds explicit interpretations to raw texts at different levels. It enables efficient retrieval of linguistic patterns, supports downstream tasks, and creates resources that can be reused across studies. Good annotation practice preserves the original data, documents annotation procedures and decisions, uses clear and consistently applicable categories, and builds on established conventions where appropriate. Annotation quality must be evaluated in terms of validity, accuracy, and consistency: whether the categories capture meaningful linguistic distinctions, whether assigned labels match reference annotations, and whether independent annotators apply the scheme similarly.

10 Semantic role labeling

References

- Jurafsky and Martin (2026) Semantic Role Labeling. In *Speech and Language Processing*.

Semantic roles provide a way of representing the participants in an event according to the roles they play with respect to a predicate. Beyond describing constituents through syntactic relations (e.g., *subject*, *object*), semantic roles capture more abstract relationships such as who performs an action, what is affected, or where an entity moves.

10.1 Semantic roles

Different theories propose different inventories of thematic roles.[21] A relatively small set of abstract semantic roles is often sufficient for computational applications. Table 10 summarizes several common roles.

Role	Definition	Example
Agent	Volitional causer of an event	*The waiter* spilled the soup.
Experiencer	Participant experiencing an event or state	*John* has a headache.
Force	Non-volitional causer of an event	*The wind* blew debris into our yards.
Theme	Participant most directly affected by an event	Benjamin Franklin broke *the ice*.
Result	End product of an event	The city built *a baseball diamond*.
Content	Content of a propositional event	Mona asked, "*You met Mary Ann at a supermarket?*"
Instrument	Instrument used in an event	He stunned them *with a shocking device*.
Beneficiary	Participant benefiting from an event	Ann made reservations *for her boss*.
Source	Origin of a transfer or movement	I flew in *from Boston*.
Goal	Destination of a transfer or movement	I drove *to Portland*.

Table 10: Examples of common semantic roles.

Semantic roles are useful because they provide a relatively shallow representation of meaning that abstracts away from some differences in surface syntax. For example, consider (1) "The librarian gave Maya a book" and (2) "The librarian gave a book to Maya." Although the grammatical structures differ, the semantic roles remain the same: the librarian is the giver, Maya is the recipient, and a book is the entity being transferred. This representation allows us to recognize the same underlying event in syntactically different expressions (which is why one application of semantic role labeling in NLP is information extraction, where the goa is to identify `whom did what to whom`, regardless of how the information is expressed.)

[21] The term thematic role typically refers to a general participant category in linguistic theory (e.g., Agent, Patient). Semantic role is a broader term that may also encompass event- or framework-specific roles (e.g., Buyer and Seller in a commercial transaction), although the two terms are often used interchangeably.

10.2 Diathesis alternations

To expand on the preceding discussion, one motivation for semantic roles is that participants can occupy different syntactic positions while maintaining the same semantic relationship to an event. Consider the following uses of the verb *melt*:

1. *Sarah*$_{\text{AGENT}}$ melted *the ice*$_{\text{THEME}}$.
2. *Sarah*$_{\text{AGENT}}$ melted *the ice*$_{\text{THEME}}$ with *a heat lamp*$_{\text{INSTRUMENT}}$.
3. *The heat lamp*$_{\text{INSTRUMENT}}$ melted *the ice*$_{\text{THEME}}$.
4. *The ice*$_{\text{THEME}}$ melted.
5. *The ice*$_{\text{THEME}}$ was melted by *Sarah*$_{\text{AGENT}}$.

Although the grammatical functions of these participants change, their semantic roles remain relatively stable. The set of thematic arguments associated with a verb is sometimes called its *thematic grid* or *case frame*. Different syntactic realizations of the same underlying argument structure are known as *verb alternations* or *diathesis alternations*. A familiar example is the dative alternation:

She gave Mary a book.
She gave a book to Mary.

Diathesis alternations are not random. Rather, verbs with similar meanings often exhibit similar patterns of argument realization. For example, verbs such as *break*, *melt*, and *freeze* can participate in the causative–inchoative alternation (e.g., *John melted the ice* vs. *The ice melted*). Building on such observations, Levin (1993) classified more than 3,000 English verbs into groups based on the syntactic alternations they exhibit. This work also influenced the development of lexical resources such as *VerbNet* (Schuler, 2005), which grouped verbs into classes based on shared syntactic patterns and semantic properties.

However, defining semantic roles consistently across different verbs is not straightforward. One challenge is that there is no universally agreed-upon set of semantic roles. Another is that broad labels (e.g., agent, theme) do not always capture the differences in how participants are involved in events. For example, the subjects of *John opened the door*, *John heard a noise*, and *John fell* are all syntactic subjects, but they differ in how much control or intention they have over the events.

Researchers have proposed different approaches to addressing these challenges. One approach is to treat semantic roles as clusters of properties rather than fixed categories. For example, Dowty's (1991) notion of *Proto-Agent* and *Proto-Patient* characterizes arguments according to properties such as volition, causation, and being affected by an event. Another approach is to define roles in relation to specific predicates or groups of semantically related predicates, rather than assuming that the same set of roles applies universally. Two major resources that adopt versions of this predicate-specific approach are *PropBank* (Palmer et al., 2005) and *FrameNet* (Baker et al., 1998).

10.3 PropBank

PropBank is a corpus in which predicates and their arguments are annotated with semantic roles. Because defining a universal inventory of thematic roles is difficult, PropBank defines roles with respect to individual verb senses. Each sense of a predicate has its own roleset. Core arguments are represented using numbered labels such as ARG0, ARG1, and so on. Their exact interpretations depend on the predicate, although some broad tendencies are common:

- ARG0: typically Proto-Agent
- ARG1: typically Proto-Patient
- ARG2: often beneficiary, instrument, attribute, or end state
- ARG3: often starting point, beneficiary, instrument, or attribute
- ARG4: often ending point

This allows sentences with different surface structures to receive comparable semantic representations:

[ARG0 Big Fruit Co.] increased [ARG1 the price of bananas].

[ARG1 The price of bananas] was increased again [ARG0 by Big Fruit Co.].

PropBank also includes modifier arguments, usually written as ARGM. Unlike numbered arguments, these labels are relatively stable across predicates.

Label	Meaning
ARGM-TMP	Time: when?
ARGM-LOC	Location: where?
ARGM-DIR	Direction: where to/from?
ARGM-MNR	Manner: how?
ARGM-PRP / ARGM-CAU	Purpose or cause: why?
ARGM-REC	Reciprocal
ARGM-ADV	Miscellaneous adverbial meaning
ARGM-PRD	Secondary predicate

Table 11: Examples of PropBank modifier roles.

10.4 FrameNet

FrameNet takes a somewhat different approach, which organizes lexical items around semantic frames. Here, a *frame* represents a structured package of background knowledge associated with a particular type of event or situation. Words such as *reservation*, *flight*, *fare*, *travel*, and *plane*, for example, can be understood through shared background knowledge about air travel.

Each frame contains a set of predicates that evoke the frame and a set of frame-specific semantic roles called *frame elements*. For example, the `Change_position_on_a_scale` frame describes situations in which an ITEM changes with respect to some ATTRIBUTE, moving from an INITIAL_VALUE to a FINAL_VALUE.

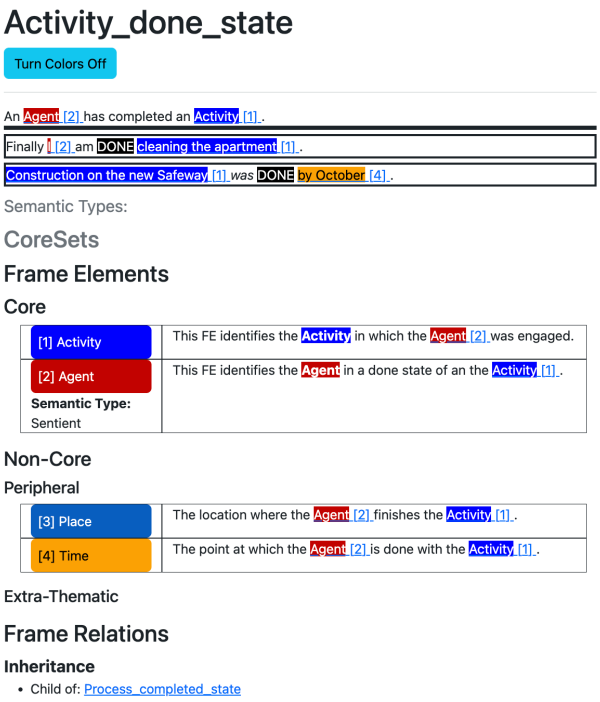


Figure 7: Example of a FrameNet frame, *Activity_done_state*. The frame defines two core frame elements, ACTIVITY and AGENT, and non-core elements such as PLACE and TIME. The annotated example sentences illustrate how these frame elements are realized in different surface forms. Adapted from FrameNet.

Similar to the approach from PropBank, FrameNet distinguishes between core frame elements, which are central to a particular frame, and non-core frame elements, which express more general information such as time, location, or manner. It also represents relationships among frames, including inheritance and causal relations, allowing broader semantic generalizations to be represented. Note that FrameNet-related resources have been developed for several languages, including Spanish, German, Japanese, Portuguese, Italian, and Chinese. More information can be found here: `https://framenet.icsi.berkeley.edu/`.

10.5 Semantic role labeling

Semantic Role Labeling (SRL) is the computational task of automatically identifying predicates and determining the semantic roles of their arguments. Traditional SRL systems are supervised machine-learning systems trained on annotated resources such as PropBank or FrameNet. These resources specify what counts as a predicate, define the possible role labels, and provide training and evaluation data.

Consider the sentence:

Sarah baked her friend a cake on Sunday.

Given *baked* as the predicate, a PropBank-style analysis (based on the *bake.01* roleset) would

assign:

- *Sarah*: ARG0 — baker
- *a cake*: ARG1 — thing created
- *her friend*: ARG3 — beneficiary
- *on Sunday*: ARGM-TMP — time

This example also illustrates the relationship between semantic roles and argument structure constructions. The verb *bake* primarily describes an activity of creating something through heat, as in *Sarah baked a cake*. However, when the verb occurs in the ditransitive construction (Subject–Verb–Indirect Object–Direct Object), as in *Sarah baked her friend a cake*, the sentence additionally conveys that the cake is intended for the friend. From a constructionist perspective, this interpretation arises from the interaction between the lexical meaning of the verb and the meaning associated with the ditransitive construction.

PropBank captures these participant relationships through predicate-specific argument labels. Importantly, the numerical labels do not necessarily correspond to the same semantic roles across predicates. For example, the beneficiary of *bake.01* is labeled ARG3, whereas the recipient of *give.01* is labeled ARG2.

10.5.1 Feature-based approaches

Earlier SRL systems commonly relied on manually designed linguistic features. A sentence was first syntactically parsed, and the resulting parse tree was traversed to identify predicates and candidate arguments. For each candidate argument, the system extracted features such as:

- phrase or constituent type;
- head word;
- position relative to the predicate;
- syntactic path between the argument and predicate; and
- properties of the predicate itself.

A supervised classifier then assigns a semantic-role label to each candidate constituent. The label set typically also includes a NONE category, since many constituents in the parse tree do not serve as arguments of the target predicate.

One limitation of treating semantic role labeling as a series of independent classification decisions is that the role assigned to one argument may depend on the roles assigned to other arguments of the same predicate. Consider the sentence *Sarah baked her friend a cake* again. Suppose a model evaluates each argument separately and predicts ARG1 for both *her friend* and *a cake*. Although these predictions may appear plausible when considered individually, they are inconsistent when considered together. For the predicate *bake.01*, ARG1 refers to the thing being created, while ARG3 identifies the beneficiary. Moreover, core argument labels such as ARG0 and ARG1

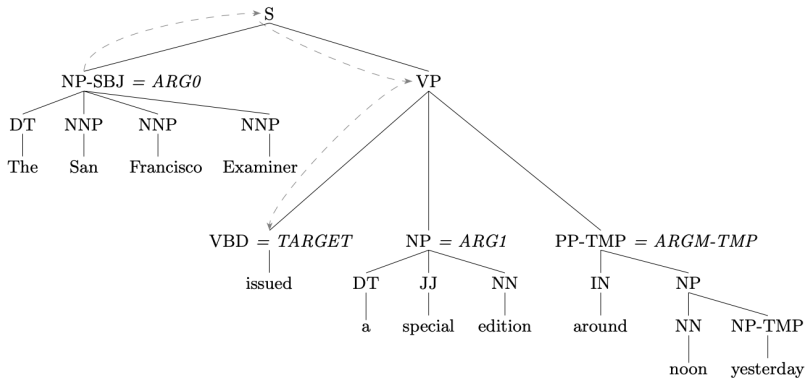


Figure 8: Parse tree of a PropBank-annotated sentence with semantic role labels. The dotted line illustrates the syntactic path feature (NP↑S↓VP↓VBD) connecting the ARG0 constituent (*The San Francisco Examiner*) to the predicate. Adapted from Jurafsky and Martin (2026), Chapter 22, p. 9.

are generally assigned at most once for a given predicate. In short, the model needs to consider the relationships among arguments, not selecting the highest-scoring label for each one independently.

One solution is to combine local predictions with global optimization. In the first stage, a model assigns scores to possible semantic roles for each candidate argument. Rather than immediately accepting the highest-scoring label for every argument, a second stage evaluates possible combinations of labels and selects the one that produces the most plausible overall analysis. For example, even if ARG1 receives a relatively high score for *her friend*, the model may ultimately assign ARG3 to this argument because *a cake* is a better candidate for ARG1. The global decision can take into account constraints such as avoiding duplicate core roles, preventing overlapping argument spans, and ensuring compatibility with the predicate's argument structure. Different computational approaches have been used for this purpose, including Viterbi decoding, reranking, and integer linear programming.

10.5.2 Neural approaches: SRL as sequence labeling

Neural approaches can formulate SRL as a sequence-labeling task, in which the model assigns a semantic role label to each token in a sentence. One common approach uses the BIO tagging scheme, where B marks the beginning of an argument span, I marks tokens inside the same span, and O marks tokens that do not belong to any argument span. For example:

- `B-ARG0`: beginning of an ARG0 span;
- `I-ARG0`: continuation of an ARG0 span;
- `B-ARG1`: beginning of an ARG1 span;
- `O`: token outside any argument span.

The prediction problem can be represented as:

$$\hat{y} = \arg\max_{y \in T} P(y \mid w),$$

where w is the input word sequence, y is one possible label sequence, T is the set of possible label sequences, and $P(y \mid w)$ is the probability assigned to a particular sequence. The predicted sequence $\hat{y}$ is therefore the sequence with the highest probability.

Suppose the sentence is:

the cats love hats

A model might consider the following possibilities:

A: `B-ARG0 I-ARG0 B-PRED B-ARG1` 0.72
B: `B-ARG0 I-ARG0 O B-ARG1` 0.18
C: `O O B-PRED B-ARG1` 0.10

Because sequence A has the highest probability, it is selected as the model's final prediction. Figure 9 illustrates how a neural model assigns semantic role labels to individual tokens. The encoder first processes the sentence to generate contextualized representations of each token. Because semantic roles are defined in relation to a particular predicate, the model also receives information about which word is being analyzed as the predicate (in this case, *love*). The resulting representations are then passed through a feedforward network and a softmax layer, which computes scores for possible semantic role labels. The model uses these scores to predict labels such as B-ARG0 and I-ARG0 for *the cats*, and B-ARG1 for *hats*.

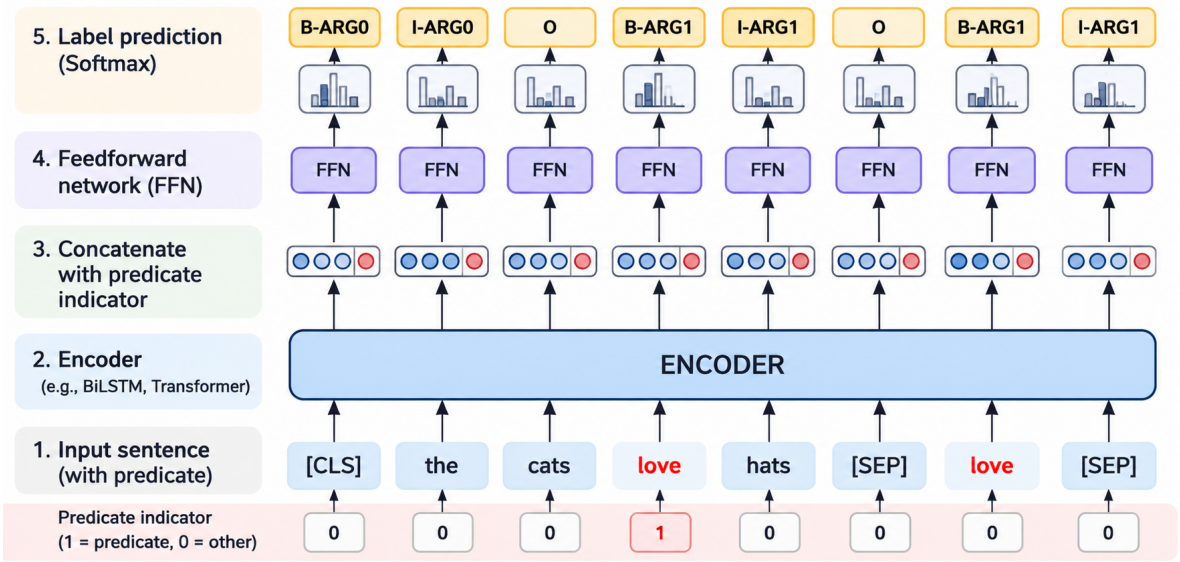


Figure 9: A neural sequence-labeling approach to SRL. Adapted from Figure 22.6 in Chapter 22 of Jurafsky and Martin (2026).

10.6 Beyond semantic role labeling

Semantic role labeling provides a useful intermediate representation between syntax and meaning, but most conventional SRL schemes represent semantic relations primarily around individual predicates. They also usually operate within a single sentence and focus on explicitly realized arguments. This creates difficulties for phenomena such as implicit arguments, where an argument is understood from context but not overtly expressed. Work on *implicit semantic role labeling* (iSRL) attempts to recover such arguments.

A broader approach to semantic representation is *Abstract Meaning Representation* (*AMR*). While SRL primarily identifies predicates and their arguments, AMR attempts to represent the meaning of an entire sentence as a structured graph. Nodes represent concepts or events, and labeled edges represent semantic relations between them. An AMR graph can therefore encode predicate–argument relations similar to those captured by SRL, while also representing additional information such as entities, modifiers, negation, quantities, and relations among events.

For example, a sentence such as:

The boy wants to go.

can be represented schematically as:

```
(w / want-01
   :ARG0 (b / boy)
   :ARG1 (g / go-01
             :ARG0 b))
```

Here, the graph represents not only the arguments of *want* and *go*, but also the fact that the same *boy* is understood as the participant associated with both events. Thus, SRL and AMR can be viewed as related but different levels of semantic representation. SRL focuses on identifying the semantic arguments of individual predicates, whereas AMR attempts to construct a more integrated representation of sentence meaning.

Chapter Summary

Semantic roles describe the relationships between predicates and their participants, providing a representation of meaning that goes beyond grammatical functions such as subjects and objects. Defining a universal set of semantic roles is challenging, leading to predicate- and frame-specific approaches such as PropBank and FrameNet. Building on these annotated resources, semantic role labeling (SRL) aims to automatically identify predicates and assign semantic roles to their arguments. Earlier feature-based approaches relied on syntactic information and manually designed features, often incorporating global optimization to ensure consistent role assignments. More recent neural approaches commonly formulate SRL as a sequence-labeling task, using contextualized representations and BIO tagging to iden-

tify argument spans. Although conventional SRL primarily focuses on explicitly expressed arguments of individual predicates, extensions such as implicit SRL address unexpressed arguments, while Abstract Meaning Representation (AMR) provides a broader graph-based representation of sentence meaning.

References

Baker, C. F., Fillmore, C. J., and Lowe, J. B. (1998). The berkeley FrameNet project. In *Proceedings of the 36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics*, pages 86–90. Association for Computational Linguistics.

Berzak, Y., Kenney, J., Spadine, C., Wang, J. X., Lam, L., Mori, K. S., Garza, S., and Katz, B. (2016). Universal Dependencies for learner English. In Erk, K. and Smith, N. A., editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pages 737–746, Berlin, Germany. Association for Computational Linguistics.

Biber, D. (1990). Methodological issues regarding corpus-based analyses of linguistic variation. *Literary and linguistic computing*, 5(4):257–269.

Biber, D. (1993). Representativeness in corpus design. *Literary and linguistic computing*, 8(4):243–257.

Biber, D., Reppen, R., Staples, S., and Egbert, J. (2020). Exploring the longitudinal development of grammatical complexity in the disciplinary writing of l2-english university students. *International Journal of Learner Corpus Research*, 6(1):38–71.

Bies, A., Mott, J., Warner, C., and Kulick, S. (2012). English web treebank. *Linguistic Data Consortium, Philadelphia, PA*.

Bulté, B. and Housen, A. (2012). Defining and operationalising l2 complexity. *Dimensions of L2 performance and proficiency: Complexity, accuracy and fluency in SLA*, 32:21.

Bulté, B., Housen, A., and Pallotti, G. (2025). Complexity and difficulty in second language acquisition: A theoretical and methodological overview. *Language Learning*, 75(2):533–574.

Choi, J. D., Tetreault, J., and Stent, A. (2015). It depends: Dependency parser comparison using a web-based evaluation tool. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing* (*Volume 1: Long Papers*), pages 387–396.

de Marneffe, M.-C., Manning, C. D., Nivre, J., and Zeman, D. (2021). Universal dependencies. *Computational linguistics*, 47(2):255–308.

DeKeyser, R. M. (2005). What makes learning second-language grammar difficult? a review of issues. *Language learning*, 55:1.

Di Nuovo, E., Bosco, C., Mazzei, A., and Sanguinetti, M. (2019). Towards an italian learner treebank in universal dependencies. In *Proceedings of the Sixth Italian Conference on Computational Linguistics* (*CLiC-it 2019*), pages 151–158, Bari, Italy. CEUR Workshop Proceedings.

Dowty, D. R. (1991). Thematic proto-roles and argument selection. *Language*, 67(3):547–619.

Dunn, J. (2022). *Natural language processing for corpus linguistics*. Cambridge University Press.

Egbert, J., Biber, D., and Gray, B. (2022). *Designing and evaluating language corpora: A practical framework for corpus representativeness*. Cambridge University Press.

Firth, J. R. (1957). A synopsis of linguistic theory, 1930–1955. In *Studies in Linguistic Analysis*, pages 1–32. Blackwell, Oxford.

Granger, S. (2015). Contrastive interlanguage analysis: A reappraisal. *International Journal of Learner Corpus Research*, 1(1):7–24.

Granger, S. (2026). Corpus-based research in second language acquisition. In Nesi, H. and Milin, P., editors, *International Encyclopedia of Language and Linguistics*. Elsevier, 3 edition.

Granger, S. and Paquot, M. (2015). Electronic lexicography goes local: Design and structures of a needs-driven online academic writing aid. *Lexicographica*, 31(1):118–141.

Granger, S. and Paquot, M. (2022). The Louvain English for Academic Purposes Dictionary: User manual. CECL Papers 5, Centre for English Corpus Linguistics, Université catholique de Louvain, Louvain-la-Neuve.

Gries, S. T. (2009). What is corpus linguistics? *Language and linguistics compass*, 3(5):1225–1241.

Gries, S. T. (2026). Corpus linguistics and psycholinguistics. In Nesi, H. and Milin, P., editors, *International Encyclopedia of Language and Linguistics*, pages 412–416. Elsevier, 3 edition.

Hartle, S. (2023). From learner corpus to data-driven learning (ddl) in eap writing. improving lexical usage in academic writing. *EuroAmerican Journal of Applied Linguistics and Languages*, 10(2):9–31.

Hayes, A. F. and Krippendorff, K. (2007). Answering the call for a standard reliability measure for coding data. *Communication methods and measures*, 1(1):77–89.

Housen, A. and Simoens, H. (2016). Introduction: Cognitive perspectives on difficulty and complexity in l2 acquisition. *Studies in Second Language Acquisition*, 38(2):163–175.

Jarvis, S. and Hashimoto, B. J. (2021). How operationalizations of word types affect measures of lexical diversity. *International Journal of Learner Corpus Research*, 7(1):163–194.

Jurafsky, D. and Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition. Online manuscript released January 6, 2026.

Kyle, K. (2021). Natural language processing for learner corpus research.

Kyle, K., Eguchi, M., Miller, A., and Sither, T. (2022). A dependency treebank of spoken second language English. In Kochmar, E., Burstein, J., Horbach, A., Laarmann-Quante, R., Madnani, N., Tack, A., Yaneva, V., Yuan, Z., and Zesch, T., editors, *Proceedings of the 17th Workshop on Innovative Use of NLP for Building Educational Applications* (*BEA 2022*), pages 39–45, Seattle, Washington. Association for Computational Linguistics.

Kyle, K. and Sung, H. (2026). Tagging and parsing. In Nesi, H. and Milin, P., editors, *International*

Encyclopedia of Language and Linguistics. Elsevier, 3 edition.

Kyle, K., Sung, H., Eguchi, M., and Zenker, F. (2024). Evaluating evidence for the reliability and validity of lexical diversity indices in l2 oral task responses. *Studies in Second Language Acquisition*, 46(1):278–299.

Larsen-Freeman, D. (1978). An esl index of development. *TESOL quarterly*, pages 439–448.

Lee, J., Leung, H., and Li, K. (2017). Towards universal dependencies for learner chinese. In *Proceedings of the NoDaLiDa 2017 Workshop on Universal Dependencies (UDW 2017)*, volume 135 of *Linköping Electronic Conference Proceedings*, pages 67–71. Linköping University Electronic Press.

Leech, G. (2004). Adding linguistic annotation. In *Developing linguistic corpora: A guide to good practice*, pages 17–29. Oxbow Books.

Lenci, A. (2018). Distributional models of word meaning. *Annual review of Linguistics*, 4:151–171.

Levin, B. (1993). *English Verb Classes and Alternations: A Preliminary Investigation*. University of Chicago Press, Chicago.

Marcinkiewicz, M. A. (1994). Building a large annotated corpus of english: The penn treebank. *Using large corpora*, 273(31):5.

McEnery, T. (2006). Corpus-based language studies: An advanced resource book. *London and New York: Routledge*.

Meurers, D. (2020). Natural language processing and language learning. *Encyclopedia of applied linguistics*, pages 4193–4205.

Meurers, D. and Dickinson, M. (2017). Evidence and interpretation in language learning research: Opportunities for collaboration with computational linguistics. *Language Learning*, 67(S1):66–95.

Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013a). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G., and Dean, J. (2013b). Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26.

Murakami, A. and Alexopoulou, T. (2016). L1 influence on the acquisition order of english grammatical morphemes: A learner corpus study. *Studies in Second Language Acquisition*, 38(3):365–401.

Nation, I. S. P. (2016). *Making and Using Word Lists for Language Learning and Testing*. John Benjamins Publishing Company.

Nation, I. S. P. and Webb, S. (2011). *Researching and Analyzing Vocabulary*. Heinle.

Nivre, J., De Marneffe, M.-C., Ginter, F., Hajic, J., Manning, C. D., Pyysalo, S., Schuster, S., Tyers, F., and Zeman, D. (2020). Universal dependencies v2: An evergrowing multilingual treebank

collection. In *Proceedings of the twelfth language resources and evaluation conference*, pages 4034–4043.

Norris, J. M. and Ortega, L. (2009). Towards an organic approach to investigating caf in instructed sla: The case of complexity. *Applied linguistics*, 30(4):555–578.

Ortega, L. (2003). Syntactic complexity measures and their relationship to l2 proficiency: A research synthesis of college-level l2 writing. *Applied linguistics*, 24(4):492–518.

Osgood, C. E., Suci, G. J., and Tannenbaum, P. H. (1957). *The Measurement of Meaning*. University of Illinois Press, Urbana, IL.

Pallotti, G. (2015). A simple view of linguistic complexity. *Second Language Research*, 31(1):117–134.

Palmer, M., Gildea, D., and Kingsbury, P. (2005). The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106.

Pulido, E., Pugh, R., and Liu, Z. (2025). I speak for the árboles: Developing a dependency treebank for spanish l2 and heritage speakers. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (*Volume 4: Student Research Workshop*), pages 814–822.

Rescher, N. (1998). *Complexity: A philosophical overview*. Transaction Publishers.

Rozovskaya, A. (2024). Universal Dependencies for learner Russian. In Calzolari, N., Kan, M.-Y., Hoste, V., Lenci, A., Sakti, S., and Xue, N., editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation* (*LREC-COLING 2024*), pages 17112–17119, Torino, Italia. ELRA and ICCL.

Santorini, B. (1990). Part-of-speech tagging guidelines for the penn treebank project.

Schmuckler, M. A. (2026). *Ecological Validity*. MIT Press. https://oecs.mit.edu/pub/1y8kpgak.

Schuler, K. K. (2005). *VerbNet: A broad-coverage, comprehensive verb lexicon*. University of Pennsylvania.

Silveira, N., Dozat, T., De Marneffe, M.-C., Bowman, S. R., Connor, M., Bauer, J., and Manning, C. D. (2014). A gold standard dependency corpus for english. In *LREC*, pages 2897–2904.

Stubbs, M. (2006). Corpus analysis: The state of the art and three types of unanswered questions. *Equinox eBooks Publishing*, pages 15–36.

Sung, H. and Kyle, K. (2025). Usage-based analysis of l2 oral proficiency: Characteristics of argument structure construction use. *Studies in Second Language Acquisition*, 47(4):991–1017.

Sung, H. and Shin, G.-H. (2024). Constructing a dependency treebank for second language learners of korean. In Calzolari, N., Kan, M.-Y., Hoste, V., Lenci, A., Sakti, S., and Xue, N., editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation* (*LREC-COLING 2024*), pages 3747–3758, Torino, Italia. ELRA and ICCL.

Sung, H. and Shin, G.-H. (2025). Towards robust morphosyntactic analysis of l2 korean: Evaluating

and fine-tuning a korean language model. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 24(11):1–21.

Teubert, W. (2005). My version of corpus linguistics. *International journal of corpus linguistics*, 10(1):1–13.

Tracy-Ventura, N. and Paquot, M. (2021). *The Routledge handbook of second language acquisition and corpora*. Routledge London and New York.

Vajjala, S., Majumder, B., Gupta, A., and Surana, H. (2020). *Practical natural language processing: a comprehensive guide to building real-world NLP systems*. O'Reilly Media.

Vilkaitė-Lozdienė, L. and Schmitt, N. (2020). Frequency as a guide for vocabulary usefulness: High-, mid-, and low-frequency words. In Webb, S., editor, *The Routledge Handbook of Vocabulary Studies*, pages 81–96. Routledge.

Volodina, E., Masciolini, A., Megyesi, B., Prentice, J., Rudebeck, L., Sundberg, G., and Wirén, M. (2025). SweLL with pride: How to put a learner corpus to good use. *Huminfra Handbook*. Forthcoming.

Webb, S. (2021). The lemma dilemma: How should words be operationalized in research and pedagogy? *Studies in Second Language Acquisition*, 43(5):941–949.

Weischedel, R., Pradhan, S., Ramshaw, L., Palmer, M., Xue, N., Marcus, M., Taylor, A., Greenberg, C., Hovy, E., Belvin, R., et al. (2022). *OntoNotes Release 5.0*. Borealis.

Zeman, D. (2025). Corpus-based language comparison: From morphology to dependencies and beyond. *Estudos Linguísticos (São Paulo. 1978)*, 54(1):259–275.